Hasil Running 6 – Natural Language Processing
Nayla Putri Cahya Ramadani – 2024071020

01_CNN_RNN_ATIS_intents

Link Google Colab:
https://colab.research.google.com/drive/1racTvsK--N4IYU5JBMIV0t0-9Lxrc334?usp=sharing

Cell 1 – Import Library

```
#general imports
import os
import sys
import random
random.seed(0) #for reproducability of results

#basic imports
import numpy as np
import pandas as pd

#NN imports
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.layers import Dense, Input, GlobalMaxPooling1D
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Embedding, LSTM
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.initializers import Constant

#encoder
from sklearn.preprocessing import LabelEncoder
```

Cell ini digunakan untuk memanggil berbagai library yang dibutuhkan dalam proses pengolahan teks dan pembuatan model. Library NumPy dan Pandas membantu mengolah data, sedangkan TensorFlow Keras menyediakan komponen untuk membangun model seperti CNN dan LSTM. Selain itu, Tokenizer dan pad_sequences digunakan untuk mengubah teks menjadi data numerik dengan panjang yang seragam, sementara LabelEncoder membantu mengubah label kategori menjadi bentuk numerik. Pengaturan random.seed(0) juga digunakan agar hasil yang melibatkan bilangan acak lebih konsisten saat program dijalankan kembali.

Cell 2 – Memuat dan Memeriksa Dataset Training

```
from Data.utils import fetch_data, read_method

sents,labels,intents = fetch_data('Data/data2/atis.train.w-intent.iob')

train_sentences = [" ".join(i) for i in sents]

train_texts = train_sentences
train_labels= intents.tolist()

vals = []

for i in range(len(train_labels)):
    if "#" in train_labels[i]:
        vals.append(i)

for i in vals[::-1]:
    train_labels.pop(i)
    train_texts.pop(i)

print ("Number of training sentences :",len(train_texts))
print ("Number of unique intents :",len(set(train_labels)))

for i in zip(train_texts[:5], train_labels[:5]):
    print(i)
```

```
Number of training sentences : 4952
Number of unique intents : 17
('i want to fly from boston at 838 am and arrive in denver at 1110 in the morning', 'atis_flight')
('what flights are available from pittsburgh to baltimore on thursday morning', 'atis_flight')
('what is the arrival time in san francisco for the 755 am flight leaving washington', 'atis_flight_time')
('cheapest airfare from tacoma to orlando', 'atis_airfare')
('round trip fares from pittsburgh to philadelphia under 1000 dollars', 'atis_airfare')
```

Cell ini digunakan untuk mengambil dataset training dari file atis.train.w-intent.iob menggunakan fungsi fetch_data(). Data yang awalnya berbentuk kumpulan kata kemudian digabungkan menjadi kalimat, sedangkan label intent disimpan sebagai target yang akan dipelajari oleh model. Program juga menghapus data yang mengandung tanda # pada labelnya agar data tersebut tidak ikut digunakan dalam pelatihan. Terakhir, program menampilkan jumlah kalimat training, jumlah intent yang berbeda, dan 5 contoh data untuk memastikan dataset berhasil dimuat dengan benar.

Cell 3 – Mengimpor Library untuk Pengolahan Teks dan Model

```
#general imports
import os
import sys
import random
random.seed(0) #for reproducability of results

#basic imports
import numpy as np
import pandas as pd

#NN imports
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.layers import Dense, Input, GlobalMaxPooling1D
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Embedding, LSTM
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras.initializers import Constant

#encoder
from sklearn.preprocessing import LabelEncoder
```

Cell ini berisi kumpulan library yang diperlukan untuk menyiapkan data dan membangun model klasifikasi teks. Library TensorFlow Keras menyediakan lapisan seperti Dense, Conv1D, MaxPooling1D, Embedding, dan LSTM yang dapat digunakan untuk mempelajari pola dalam kalimat. Sementara itu, Tokenizer dan pad_sequences membantu mengubah teks menjadi urutan angka yang dapat diproses oleh model. LabelEncoder digunakan untuk mengubah label intent menjadi representasi numerik agar lebih mudah dipelajari oleh model.

Cell 4 – Menentukan Konfigurasi Pengolahan Data

```
MAX_SEQUENCE_LENGTH = 300
MAX_NUM_WORDS = 20000
EMBEDDING_DIM = 100
VALIDATION_SPLIT = 0.3
```

Cell ini digunakan untuk menentukan pengaturan awal sebelum data teks diproses oleh model. MAX_SEQUENCE_LENGTH = 300 menetapkan panjang maksimal setiap urutan teks menjadi 300 token, sedangkan MAX_NUM_WORDS = 20000 membatasi jumlah kata yang akan dipertimbangkan. Selanjutnya, EMBEDDING_DIM = 100 menentukan ukuran representasi vektor setiap kata, dan VALIDATION_SPLIT = 0.3 menetapkan 30% data training untuk digunakan sebagai data validasi.

Cell 5 - Mengubah Teks Menjadi Urutan Angka

```
tokenizer = Tokenizer(num_words=MAX_NUM_WORDS)
tokenizer.fit_on_texts(train_texts)
train_sequences = tokenizer.texts_to_sequences(train_texts) #Converting text to a vector of word indexes
test_sequences = tokenizer.texts_to_sequences(test_texts)
word_index = tokenizer.word_index
print('Found %s unique tokens.' % len(word_index))
```

```
Found 897 unique tokens.
```

Cell ini digunakan untuk mengubah kalimat menjadi urutan angka agar dapat diproses oleh model deep learning. Tokenizer mempelajari kosakata dari data training melalui fit_on_texts(), kemudian texts_to_sequences() mengubah teks training dan testing menjadi indeks angka berdasarkan kata-katanya. Program juga menyimpan pemetaan kata ke indeks dalam variabel word_index dan menampilkan jumlah kata unik yang ditemukan. Berdasarkan output, terdapat 897 token unik dalam kosakata yang berhasil dipelajari.

Cell 6 - Mengubah Label Intent Menjadi Angka

```
le = LabelEncoder()
le.fit(train_labels)
train_labels = le.transform(train_labels)
test_labels = le.transform(test_labels)
```

Cell ini digunakan untuk mengubah label intent yang berbentuk teks menjadi angka agar bisa dipelajari oleh model klasifikasi. LabelEncoder terlebih dahulu mempelajari kategori label dari data training melalui fit(), kemudian transform() mengubah label training dan testing menjadi representasi numerik. Dengan proses ini, setiap kategori intent memiliki kode angka yang konsisten. Hasilnya, label siap digunakan dalam tahap persiapan data sebelum pelatihan model.

Cell 7 - Menyamakan Panjang Teks dan Membagi Dataset

```
#Converting this to sequences to be fed into neural network. Max seq. len is 1000 as set earlier
 #initial padding of 0s, until vector is of size MAX_SEQUENCE_LENGTH
trainvalid_data = pad_sequences(train_sequences, maxlen=MAX_SEQUENCE_LENGTH)
test_data = pad_sequences(test_sequences, maxlen=MAX_SEQUENCE_LENGTH)
trainvalid_labels = to_categorical(train_labels)

test_labels = to_categorical(np.asarray(test_labels), num_classes= trainvalid_labels.shape[1])

# split the training data into a training set and a validation set
indices = np.arange(trainvalid_data.shape[0])
np.random.shuffle(indices)
trainvalid_data = trainvalid_data[indices]
trainvalid_labels = trainvalid_labels[indices]
num_validation_samples = int(VALIDATION_SPLIT * trainvalid_data.shape[0])
x_train = trainvalid_data[:-num_validation_samples]
y_train = trainvalid_labels[:-num_validation_samples]
x_val = trainvalid_data[-num_validation_samples:]
y_val = trainvalid_labels[-num_validation_samples:]
#This is the data we will use for CNN and RNN training
print('Splitting the train data into train and valid is done')
```

```
Splitting the train data into train and valid is done
```

Cell ini digunakan untuk menyamakan panjang setiap urutan teks menggunakan pad_sequences(), sehingga seluruh data memiliki panjang maksimal 300 token sesuai konfigurasi sebelumnya. Label intent kemudian diubah menjadi format kategorikal menggunakan to_categorical() agar sesuai dengan kebutuhan klasifikasi multikelas. Selanjutnya, data training diacak dan dibagi menjadi data training serta data validasi berdasarkan proporsi 70:30. Data yang sudah disiapkan ini nantinya digunakan untuk melatih dan mengevaluasi model CNN maupun RNN, termasuk LSTM.

Cell 8 - Mengunduh Dataset GloVe 6B

```
!mkdir -p Data/glove.6B
!wget http://nlp.stanford.edu/data/glove.6B.zip
!unzip glove.6B.zip -d Data/glove.6B
```

```
--2026-10-10 20:16:07--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-10-10 20:16:07--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-10-10 20:16:07--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: ‘glove.6B.zip’

glove.6B.zip        100%[===================>] 822.24M  4.99MB/s    in 2m 51s

2026-10-10 20:18:59 (4.80 MB/s) - ‘glove.6B.zip’ saved [862182613/862182613]

Archive:  glove.6B.zip
  inflating: Data/glove.6B/glove.6B.50d.txt
  inflating: Data/glove.6B/glove.6B.100d.txt
  inflating: Data/glove.6B/glove.6B.200d.txt
  inflating: Data/glove.6B/glove.6B.300d.txt
```

Cell ini digunakan untuk mengunduh dataset GloVe 6B yang berisi representasi kata dalam bentuk vektor numerik. Perintah mkdir -p digunakan untuk membuat folder penyimpanan, sedangkan wget mengunduh file GloVe dari situs Stanford. Setelah proses unduhan selesai, perintah unzip mengekstrak file agar dapat digunakan dalam program. Dataset ini nantinya membantu model memahami hubungan antar kata berdasarkan representasi vektornya.

Cell 9 - Menyiapkan Embedding Matrix dari GloVe

```
print('Preparing embedding matrix.')
# Download GloVe 6B from here: https://nlp.stanford.edu/projects/glove/
BASE_DIR = 'Data'
GLOVE_DIR = os.path.join(BASE_DIR, 'glove.6B')
embeddings_index = {}
with open(os.path.join(GLOVE_DIR, 'glove.6B.100d.txt'), encoding="utf-8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs

print('Found %s word vectors in Glove embeddings.' % len(embeddings_index))
#print(embeddings_index["google"])

# prepare embedding matrix - rows are the words from word_index, columns are the embeddings of that word from glove.
num_words = min(MAX_NUM_WORDS, len(word_index)) + 1
embedding_matrix = np.zeros((num_words, EMBEDDING_DIM))
for word, i in word_index.items():
    if i > MAX_NUM_WORDS:
        continue
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is not None:
        # words not found in embedding index will be all-zeros.
        embedding_matrix[i] = embedding_vector
embedding_layer = Embedding(num_words,
                            EMBEDDING_DIM,
                            embeddings_initializer=Constant(embedding_matrix),
                            input_length=MAX_SEQUENCE_LENGTH,
                            trainable=False)
print("Preparing of embedding matrix is done")
```

```
Preparing embedding matrix.
Found 400000 word vectors in Glove embeddings.
Preparing of embedding matrix is done
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
```

Cell ini digunakan untuk membaca vektor kata dari GloVe 6B dan mencocokkannya dengan kosakata yang telah dibuat menggunakan tokenizer. Setiap kata yang ditemukan akan disimpan bersama vektor numeriknya ke dalam embeddings_index, kemudian digunakan untuk membentuk embedding_matrix dengan dimensi vektor 100. Matriks tersebut dimasukkan ke dalam layer Embedding dengan trainable=False, sehingga bobot embedding dari GloVe tetap digunakan tanpa diperbarui selama pelatihan model. Berdasarkan output, program berhasil menemukan 400.000 vektor kata dan menyelesaikan persiapan embedding matrix.

Cell 10 - Membangun dan Melatih Model CNN dengan GloVe

```
print('Define a 1D CNN model.')

cnnmodel = Sequential()
cnnmodel.add(embedding_layer)
cnnmodel.add(Conv1D(128, 5, activation='relu'))
cnnmodel.add(MaxPooling1D(5))
cnnmodel.add(Conv1D(128, 5, activation='relu'))
cnnmodel.add(MaxPooling1D(5))
cnnmodel.add(Conv1D(128, 5, activation='relu'))
cnnmodel.add(GlobalMaxPooling1D())
cnnmodel.add(Dense(128, activation='relu'))
cnnmodel.add(Dense(len(trainvalid_labels[0]), activation='softmax'))

cnnmodel.compile(loss='categorical_crossentropy',
              optimizer='rmsprop',
              metrics=['accuracy'])

cnnmodel.build(input_shape=(None, MAX_SEQUENCE_LENGTH))

cnnmodel.summary()

#Train the model. Tune to validation set.
cnnmodel.fit(x_train, y_train,
          batch_size=128,
          epochs=1, validation_data=(x_val, y_val))
#Evaluate on test set:
score, acc = cnnmodel.evaluate(test_data, test_labels)
print('Test accuracy with CNN:', acc)
```

```
Define a 1D CNN model.
Model: "sequential_5"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 300, 100)	89,800
conv1d_9 (Conv1D)	(None, 296, 128)	64,128
max_pooling1d_6 (MaxPooling1D)	(None, 59, 128)	0
conv1d_10 (Conv1D)	(None, 55, 128)	82,048
max_pooling1d_7 (MaxPooling1D)	(None, 11, 128)	0
conv1d_11 (Conv1D)	(None, 7, 128)	82,048
global_max_pooling1d_3 (GlobalMaxPooling1D)	(None, 128)	0
dense_8 (Dense)	(None, 128)	16,512
dense_9 (Dense)	(None, 17)	2,193

```
Total params: 336,729 (1.28 MB)
Trainable params: 246,929 (964.57 KB)
Non-trainable params: 89,800 (350.78 KB)
28/28 ━━━━━━━━━━━━━━━━━━━━ 6s 148ms/step - accuracy: 0.7170 - loss: 1.1542 - val_accuracy: 0.7764 - val_loss: 0.8790
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 22ms/step - accuracy: 0.7374 - loss: 1.0288
Test accuracy with CNN: 0.7374429106712341
```

Cell ini digunakan untuk membangun model Convolutional Neural Network (CNN) yang memanfaatkan embedding dari GloVe sebagai representasi awal kata. Model terdiri dari layer Conv1D untuk mengenali pola dalam urutan teks, MaxPooling1D untuk mengurangi ukuran fitur, dan GlobalMaxPooling1D untuk mengambil fitur yang paling menonjol. Selanjutnya, layer Dense digunakan untuk mempelajari pola klasifikasi dan menghasilkan prediksi kategori intent melalui aktivasi softmax. Model dilatih menggunakan data training selama satu epoch, lalu dievaluasi menggunakan data testing untuk mengetahui nilai akurasinya.

Cell 11 - Membangun dan Melatih CNN dengan Embedding

```
print("Defining and training a CNN model, training embedding layer on the fly instead of using pre-trained embeddings")
cnnmodel = Sequential()
cnnmodel.add(Embedding(MAX_NUM_WORDS, 128, input_length=MAX_SEQUENCE_LENGTH))
cnnmodel.add(Conv1D(128, 5, activation='relu'))
cnnmodel.add(MaxPooling1D(5))
cnnmodel.add(Conv1D(128, 5, activation='relu'))
cnnmodel.add(MaxPooling1D(5))
cnnmodel.add(Conv1D(128, 5, activation='relu'))
cnnmodel.add(GlobalMaxPooling1D())
cnnmodel.add(Dense(128, activation='relu'))
cnnmodel.add(Dense(len(trainvalid_labels[0]), activation='softmax'))

cnnmodel.compile(loss='categorical_crossentropy',
              optimizer='rmsprop',
              metrics=['accuracy'])

cnnmodel.build(input_shape=(None, MAX_SEQUENCE_LENGTH))

cnnmodel.summary()

#Train the model. Tune to validation set.
cnnmodel.fit(x_train, y_train,
          batch_size=128,
          epochs=1, validation_data=(x_val, y_val))
#Evaluate on test set:
score, acc = cnnmodel.evaluate(test_data, test_labels)
print('Test accuracy with CNN:', acc)
```

```
Defining and training a CNN model, training embedding layer on the fly instead of using pre-trained embeddings
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 300, 128)	2,560,000
conv1d_6 (Conv1D)	(None, 296, 128)	82,048
max_pooling1d_4 (MaxPooling1D)	(None, 59, 128)	0
conv1d_7 (Conv1D)	(None, 55, 128)	82,048
max_pooling1d_5 (MaxPooling1D)	(None, 11, 128)	0
conv1d_8 (Conv1D)	(None, 7, 128)	82,048
global_max_pooling1d_2 (GlobalMaxPooling1D)	(None, 128)	0
dense_6 (Dense)	(None, 128)	16,512
dense_7 (Dense)	(None, 17)	2,193

```
Total params: 2,824,849 (10.78 MB)
Trainable params: 2,824,849 (10.78 MB)
Non-trainable params: 0 (0.00 B)
28/28 ━━━━━━━━━━━━━━━━━━━━ 19s 445ms/step - accuracy: 0.7087 - loss: 1.5782 - val_accuracy: 0.7556 - val_loss: 1.0315
28/28 ━━━━━━━━━━━━━━━━━━━━ 1s 21ms/step - accuracy: 0.7215 - loss: 1.2806
Test accuracy with CNN: 0.7214611768722534
```

Cell ini digunakan untuk membangun model CNN kedua sebagai pembanding dengan model yang menggunakan embedding GloVe. Pada model ini, layer Embedding dibuat langsung di dalam jaringan sehingga representasi kata dipelajari dari data training, bukan menggunakan vektor GloVe yang sudah tersedia. Arsitektur selanjutnya menggunakan beberapa layer Conv1D, MaxPooling1D, dan GlobalMaxPooling1D untuk mengekstraksi pola teks sebelum menghasilkan prediksi intent melalui softmax. Setelah model dilatih selama satu epoch, performanya dievaluasi pada data testing menggunakan akurasi agar hasilnya dapat dibandingkan dengan model CNN sebelumnya.

02_BERT_ATIS
Link Google Colab:
https://colab.research.google.com/drive/1P3a-IliXDLbO3CV6RZSWUhrEB_HKrTVt?usp=sharing

Cell 1 – Instalasi Library

```
!pip install -q transformers datasets scikit-learn
```

Cell ini digunakan untuk menginstal library yang dibutuhkan dalam proses klasifikasi teks menggunakan BERT. Library transformers digunakan untuk mengakses model BERT, datasets untuk mengambil dataset, dan scikit-learn untuk membantu proses pengolahan data serta pembagian dataset. Opsi -q digunakan agar proses instalasi tidak menampilkan terlalu banyak pesan di output.

Cell 2 – Import Library dan Setup Perangkat

```
#importing a few necessary packages
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, SequentialSampler
from torch.optim import AdamW
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from datasets import load_dataset
from transformers import BertTokenizer, BertForSequenceClassification, get_linear_schedule_with_warmup
from tqdm.auto import tqdm, trange
%matplotlib inline

# specify GPU device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Device: cuda
```

Cell ini mengimpor berbagai library yang dibutuhkan untuk mengolah data, membangun model, dan menampilkan grafik. PyTorch digunakan untuk mengelola tensor serta melatih model, sedangkan LabelEncoder membantu mengubah label teks menjadi angka. Pada bagian akhir, program memeriksa apakah GPU tersedia.

Cell 3 – Upload Dataset ATIS

```
dataset = load_dataset("tuetschek/atis")
print(dataset)
print(dataset["train"][0])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
atis_train.csv: 100% 850k/850k [00:00<00:00, 35.2MB/s]
atis_test.csv: 100% 144k/144k [00:00<00:00, 11.9MB/s]
Generating train split: 100% 4978/4978 [00:00<00:00, 77600.42 examples/s]
Generating test split: 100% 893/893 [00:00<00:00, 38503.19 examples/s]
DatasetDict({
    train: Dataset({
        features: ['id', 'intent', 'text', 'slots'],
        num_rows: 4978
    })
    test: Dataset({
        features: ['id', 'intent', 'text', 'slots'],
        num_rows: 893
    })
})
{'id': 0, 'intent': 'flight', 'text': 'i want to fly from boston at 838 am and arrive in denver at 1110 in the morning', 'slots': 'O O O O O B-fromloc.city_name O B-depart_time.time
```

Cell ini digunakan untuk mengambil dataset ATIS melalui fungsi load_dataset() dari library datasets. Dataset ATIS berisi kalimat yang berkaitan dengan permintaan informasi penerbangan beserta label

intent atau maksud dari kalimat tersebut. Perintah print() digunakan untuk melihat struktur dataset dan contoh data pertama agar format kolomnya dapat diperiksa sebelum diproses lebih lanjut.

Cell 4 – Pembersihan Dataset

```
def clean(split):
    texts = list(split["text"])
    labels = list(split["intent"])
    keep = [i for i, l in enumerate(labels) if "#" not in l]
    return [texts[i] for i in keep], [labels[i] for i in keep]
```

Cell ini mendefinisikan fungsi clean() untuk mengambil teks dan label intent dari dataset. Fungsi tersebut juga menyaring data yang labelnya mengandung tanda #, sesuai dengan logika yang digunakan pada kode. Setelah proses penyaringan, fungsi mengembalikan dua daftar, yaitu daftar kalimat dan daftar label yang akan digunakan dalam pelatihan maupun pengujian model.token agar seluruh input memiliki ukuran yang seragam. Hasilnya disimpan dalam bentuk tensor berupa input_ids, attention_masks, dan labels untuk digunakan pada tahap pelatihan model.

Cell 5 – Memuat Data Training

```
print("Loading the training Data")

train_texts, train_labels = clean(dataset["train"])

print("Number of training sentences :", len(train_texts))
print("Number of unique intents :", len(set(train_labels)))

for i in zip(train_texts[:5], train_labels[:5]):
    print(i)
```

```
Loading the training Data
Number of training sentences : 4978
Number of unique intents : 22
('i want to fly from boston at 838 am and arrive in denver at 1110 in the morning', 'flight')
('what flights are available from pittsburgh to baltimore on thursday morning', 'flight')
('what is the arrival time in san francisco for the 755 am flight leaving washington', 'flight_time')
('cheapest airfare from tacoma to orlando', 'airfare')
('round trip fares from pittsburgh to philadelphia under 1000 dollars', 'airfare')
```

Cell ini memproses bagian training data dengan memanggil fungsi clean() pada dataset pelatihan. Program kemudian menampilkan jumlah kalimat pelatihan dan jumlah label intent unik yang ditemukan. Lima contoh pertama juga ditampilkan untuk memperlihatkan hubungan antara kalimat yang digunakan dan label intent-nya.

Cell 6 – Memuat Data Testing

```
test_texts, test_labels = clean(dataset["test"])

new_labels = set(test_labels) - set(train_labels)
print("Label yang hanya ada di test:", new_labels)
keep = [i for i, l in enumerate(test_labels) if l not in new_labels]
test_texts = [test_texts[i] for i in keep]
test_labels = [test_labels[i] for i in keep]

print("Number of testing sentences :", len(test_texts))
print("Number of unique intents :", len(set(test_labels)))

for i in zip(test_texts[:5], test_labels[:5]):
    print(i)
```

```
Label yang hanya ada di test: {'flight+airline', 'day_name', 'airfare+flight', 'flight_no+airline'}
Number of testing sentences : 888
Number of unique intents : 16
('i would like to find a flight from charlotte to las vegas that makes a stop in st. louis', 'flight')
('on april first i need a ticket from tacoma to san jose departing before 7 am', 'airfare')
('on april first i need a flight going from phoenix to san diego', 'flight')
('i would like a flight traveling one way from phoenix to san diego on april first', 'flight')
('i would like a flight from orlando to salt lake city for april first on delta airlines', 'flight')
```

Cell ini mengambil dan membersihkan data pengujian menggunakan fungsi clean(). Selanjutnya, program memeriksa label yang terdapat pada data pengujian tetapi tidak ditemukan pada data pelatihan, kemudian menghapus data yang memiliki label tersebut agar proses evaluasi dapat menggunakan kelas yang dikenal oleh model. Setelah itu, program menampilkan jumlah kalimat pengujian, jumlah label unik, dan lima contoh data pertama.

Cell 7 – Mengubah Label Menjadi Angka

```
df = pd.DataFrame(data=zip(train_labels, train_texts), columns=['Labels', "Text"])

lb_make = LabelEncoder()
df["Labels"] = lb_make.fit_transform(df["Labels"])

train_texts = list(df['Text'])
train_labels = list(df['Labels'])
NUM_LABELS = len(lb_make.classes_)
print("Number of classes:", NUM_LABELS)
print("Length of training labels:", len(train_labels))
print("Length of training texts:", len(train_texts))
```

```
Number of classes: 22
Length of training labels: 4978
Length of training texts: 4978
```

Cell ini menggunakan DataFrame dari Pandas untuk menggabungkan kalimat dan label pelatihan ke dalam sebuah tabel. Selanjutnya, LabelEncoder mengubah label intent yang berbentuk teks menjadi angka agar dapat diproses oleh model BERT. Variabel NUM_LABELS digunakan untuk menghitung jumlah kelas secara otomatis, kemudian program menampilkan jumlah kelas dan panjang data teks serta label.

Cell 8 – Tokenisasi dan Padding



Cell ini mengubah kalimat menjadi token menggunakan BertTokenizer dari model bert-base-uncased. Panjang maksimum setiap kalimat ditetapkan sebanyak 128 token, sedangkan truncation=True digunakan untuk memotong kalimat yang terlalu panjang dan padding="max_length" untuk menyamakan panjang setiap input. Hasilnya disimpan dalam input_ids dan attention_masks, lalu program menampilkan token dari kalimat pertama sebagai contoh hasil tokenisasi.

Cell 9 – Menyimpan Label Training

```
intent_data_label_train = train_labels
```

Cell ini menyimpan label pelatihan ke dalam variabel intent_data_label_train. Label tersebut sebelumnya sudah diubah menjadi angka menggunakan LabelEncoder, sehingga dapat digunakan sebagai target dalam proses pelatihan model. Penggunaan variabel baru ini bertujuan untuk memperjelas bahwa data yang disimpan merupakan label yang digunakan untuk training.

Cell 10 – Membagi data dan Membuat DataLoader

```
labels_tensor = torch.tensor(intent_data_label_train)
train_idx, val_idx = train_test_split(np.arange(len(labels_tensor)),
                                      random_state=2020,
                                      test_size=0.1)

train_inputs, validation_inputs = input_ids[train_idx], input_ids[val_idx]
train_masks, validation_masks = attention_masks[train_idx], attention_masks[val_idx]
train_labels, validation_labels = labels_tensor[train_idx], labels_tensor[val_idx]

batch_size = 32

# Create an iterator of our data with torch DataLoader
train_data = TensorDataset(train_inputs, train_masks, train_labels)
train_sampler = RandomSampler(train_data)
train_dataloader = DataLoader(train_data, sampler=train_sampler, batch_size=batch_size)
validation_data = TensorDataset(validation_inputs, validation_masks, validation_labels)
validation_sampler = SequentialSampler(validation_data)
validation_dataloader = DataLoader(validation_data, sampler=validation_sampler, batch_size=batch_size)
```

Cell ini membagi data pelatihan menjadi data training dan data validasi dengan perbandingan 90:10. Fungsi train_test_split() digunakan untuk menentukan indeks pembagian data, kemudian indeks tersebut diterapkan pada input, attention mask, dan label agar semuanya tetap sesuai. Setelah itu, TensorDataset dan DataLoader digunakan untuk mengelompokkan data ke dalam batch berukuran 32, sehingga data dapat diproses secara bertahap selama pelatihan dan validasi.

Cell 11 –Memuat Model BERT

```
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=NUM_LABELS)
model.to(device)
```

```
cls.predictions.bias                      | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight    | UNEXPECTED |
cls.seq_relationship.weight               | UNEXPECTED |
cls.predictions.transform.dense.bias      | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias  | UNEXPECTED |
classifier.bias                           | MISSING    |
classifier.weight                         | MISSING    |

Notes:
- UNEXPECTED:    can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
```

```
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (dropout): Dropout(p=0.1, inplace=False)
  (classifier): Linear(in_features=768, out_features=22, bias=True)
)
```

Cell ini memuat model BertForSequenceClassification dengan model dasar bert-base-uncased untuk melakukan klasifikasi teks. Jumlah kelas ditentukan berdasarkan nilai NUM_LABELS, sehingga model dapat menyesuaikan jumlah kelas dengan label yang tersedia pada dataset. Perintah model.to(device) kemudian memindahkan model ke GPU jika tersedia atau ke CPU jika GPU tidak tersedia.

Cell 12 – Evaluasi Model pada Data Testing

```
FULL_FINETUNING = True
if FULL_FINETUNING:
    param_optimizer = list(model.named_parameters())
    no_decay = ['bias', 'LayerNorm.weight']
    optimizer_grouped_parameters = [
        {'params': [p for n, p in param_optimizer if not any(nd in n for nd in no_decay)],
         'weight_decay': 0.01},
        {'params': [p for n, p in param_optimizer if any(nd in n for nd in no_decay)],
         'weight_decay': 0.0}
    ]
else:
    param_optimizer = list(model.classifier.named_parameters())
    optimizer_grouped_parameters = [{"params": [p for n, p in param_optimizer]}]

# Number of training epochs
epochs = 4

optimizer = AdamW(optimizer_grouped_parameters, lr=3e-5)
total_steps = len(train_dataloader) * epochs
scheduler = get_linear_schedule_with_warmup(optimizer,
                                            num_warmup_steps=int(0.1 * total_steps),
                                            num_training_steps=total_steps)

# Function to calculate the accuracy of our predictions vs labels
def flat_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

# Store our loss for plotting
train_loss_set = []
```

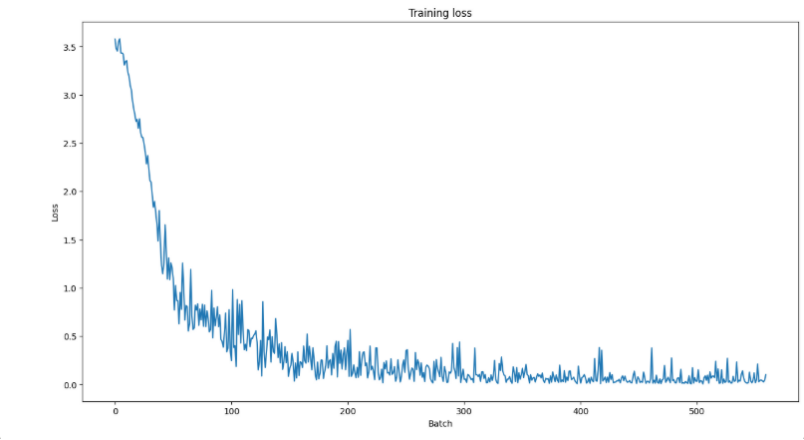
```python
# BERT training loop
for _ in trange(epochs, desc="Epoch"):

    ###### TRAINING ######
    model.train()
    tr_loss = 0
    nb_tr_examples, nb_tr_steps = 0, 0
    for step, batch in enumerate(train_dataloader):
        batch = tuple(t.to(device) for t in batch)
        b_input_ids, b_input_mask, b_labels = batch
        optimizer.zero_grad()
        # Forward pass
        outputs = model(b_input_ids, token_type_ids=None,
                        attention_mask=b_input_mask, labels=b_labels)
        loss = outputs.loss
        train_loss_set.append(loss.item())
        # Backward pass
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()
        tr_loss += loss.item()
        nb_tr_examples += b_input_ids.size(0)
        nb_tr_steps += 1
    print("Train loss: {}".format(tr_loss / nb_tr_steps))

    ###### VALIDATION ######
    model.eval()
    eval_accuracy = 0
    nb_eval_steps = 0
    for batch in validation_dataloader:
        batch = tuple(t.to(device) for t in batch)
        b_input_ids, b_input_mask, b_labels = batch
        with torch.no_grad():
            outputs = model(b_input_ids, token_type_ids=None, attention_mask=b_input_mask)
            outputs = model(b_input_ids, token_type_ids=None, attention_mask
        logits = outputs.logits.detach().cpu().numpy()
        label_ids = b_labels.to('cpu').numpy()
        eval_accuracy += flat_accuracy(logits, label_ids)
        nb_eval_steps += 1
    print("Validation Accuracy: {}".format(eval_accuracy / nb_eval_steps))

# plot training performance
plt.figure(figsize=(15, 8))
plt.title("Training loss")
plt.xlabel("Batch")
plt.ylabel("Loss")
plt.plot(train_loss_set)
plt.show()
```

```
Epoch: 100% 4/4 [06:02<00:00, 90.57s/it]
Train loss: 1.2471850349434785
Validation Accuracy: 0.96875
Train loss: 0.19992587780579923
Validation Accuracy: 0.990234375
Train loss: 0.09452690580261074
Validation Accuracy: 0.9921875
Train loss: 0.05937170041392424
Validation Accuracy: 0.9921875
```



Cell ini menjalankan proses fine-tuning BERT selama empat epoch menggunakan optimizer AdamW dengan learning rate sebesar 3e-5. Pada setiap epoch, model mempelajari data training, menghitung nilai loss, memperbarui parameter, lalu mengevaluasi hasil prediksi menggunakan data validasi untuk memperoleh validation accuracy. Program juga menampilkan nilai training loss dan validation accuracy pada setiap epoch, kemudian membuat grafik training loss untuk memperlihatkan perubahan tingkat kesalahan selama proses pelatihan.

Cell 13 – Evaluasi Model pada Data Testing

```
test_enc = tokenizer(test_texts, max_length=MAX_LEN, truncation=True,
                     padding="max_length", return_tensors="pt")
test_y = torch.tensor(lb_make.transform(test_labels))
test_data = TensorDataset(test_enc["input_ids"], test_enc["attention_mask"], test_y)
test_loader = DataLoader(test_data, sampler=SequentialSampler(test_data), batch_size=32)

model.eval()
correct, total = 0, 0
for b_ids, b_mask, b_y in test_loader:
    b_ids, b_mask = b_ids.to(device), b_mask.to(device)
    with torch.no_grad():
        logits = model(b_ids, attention_mask=b_mask).logits
    preds = logits.argmax(dim=1).cpu()
    correct += (preds == b_y).sum().item()
    total += len(b_y)
print("Test accuracy:", correct / total)
```

```
Test accuracy: 0.9707207207207207
```

Cell ini menguji model yang sudah dilatih menggunakan data pengujian yang sebelumnya belum digunakan untuk memperbarui parameter model. Kalimat pada data testing terlebih dahulu diubah menjadi token dengan pengaturan yang sama seperti data training, kemudian labelnya dikonversi menjadi angka menggunakan LabelEncoder yang sudah dilatih sebelumnya. Model menghasilkan prediksi untuk setiap kalimat, lalu jumlah prediksi yang benar dibandingkan dengan jumlah seluruh data untuk menghitung dan menampilkan nilai Test accuracy.

03_BERT_ATIS_BINARY
Link Google Colab:
https://colab.research.google.com/drive/1d2XdTTqakSBe0084Tc59hFkUUq9iSZDP?usp=sharing

Cell 1 – Instalasi Library

```
import warnings
warnings.filterwarnings("ignore")

# install
!pip install -q transformers

try:
    from google.colab import files
except ModuleNotFoundError:
    print("Not Using Colab")

DATA_DIR = "."
import os
import io
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
%matplotlib inline

# BERT imports
import torch
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, SequentialSampler
from sklearn.model_selection import train_test_split
from transformers import BertTokenizer, BertForSequenceClassification, get_linear_schedule_with_warmup
from tqdm import tqdm, trange

# replacement for keras pad_sequences (no TensorFlow/Keras needed)
def pad_sequences(sequences, maxlen, dtype="long", truncating="post", padding="post"):
    out = np.zeros((len(sequences), maxlen), dtype=np.int64)
    for i, s in enumerate(sequences):
        s = list(s)[:maxlen]
        out[i, :len(s)] = s
    return out

# specify GPU device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
if torch.cuda.is_available():
    n_gpu = torch.cuda.device_count()
    print("GPU:", torch.cuda.get_device_name(0))
else:
    print("WARNING: no GPU found, training will be very slow")
```

```
GPU: Tesla T4
```

Cell ini digunakan untuk mempersiapkan berbagai library yang dibutuhkan selama praktikum. Library Transformers dan PyTorch digunakan untuk memuat serta melatih model BERT, sedangkan NumPy dan Pandas membantu mengolah data. Selain itu, Matplotlib digunakan untuk membuat grafik, sementara Scikit-learn membantu membagi dataset menjadi data pelatihan dan validasi. Cell ini juga memeriksa ketersediaan GPU agar proses pelatihan dapat berjalan lebih cepat.

Cell 2 – Mengunduh Dataset ATIS

```
# download the ATIS dataset (pickle format) from GitHub
import urllib.request

BASE_URL = "https://raw.githubusercontent.com/howl-anderson/ATIS_dataset/master/data/raw_data/ms-cntk-atis/"

for fname in ["atis.train.pkl", "atis.test.pkl"]:
    if not os.path.exists(os.path.join(DATA_DIR, fname)):
        print("Downloading", fname)
        urllib.request.urlretrieve(BASE_URL + fname, os.path.join(DATA_DIR, fname))
    size = os.path.getsize(os.path.join(DATA_DIR, fname))
    print(fname, size, "bytes")
    assert size > 10000, (fname + " looks too small, the download probably failed. "
                          "Download it manually and upload it with files.upload().")
```

```
Downloading atis.train.pkl
atis.train.pkl 1566965 bytes
Downloading atis.test.pkl
atis.test.pkl 277306 bytes
```

Cell ini digunakan untuk mengunduh dataset ATIS dari GitHub dalam format Pickle. Terdapat dua file yang digunakan, yaitu atis.train.pkl untuk data pelatihan dan atis.test.pkl untuk data pengujian. Sebelum

mengunduh file, program memeriksa apakah file tersebut sudah tersedia di direktori penyimpanan. Ukuran file juga diperiksa untuk memastikan proses pengunduhan berjalan dengan baik.

Cell 3 – Membuat Fungsi untuk Memuat Dataset

```
# load Pickle file
def load_ds(fname, verbose=True):
    with open(fname, 'rb') as stream:
        ds,dicts = pickle.load(stream)
    if verbose:
        print('Done  loading: ', fname)
        print('      samples: {:4d}'.format(len(ds['query'])))
        print('   vocab_size: {:4d}'.format(len(dicts['token_ids'])))
        print('   slot count: {:4d}'.format(len(dicts['slot_ids'])))
        print(' intent count: {:4d}'.format(len(dicts['intent_ids'])))
    return ds,dicts

# convert Pickle file to arrays
def load_atis(filename, add_start_end_token=False, verbose=True):
    train_ds, dicts = load_ds(os.path.join(DATA_DIR,filename), verbose)
    t2i, s2i, in2i = map(dicts.get, ['token_ids', 'slot_ids','intent_ids'])
    i2t, i2s, i2in = map(lambda d: {d[k]:k for k in d.keys()}, [t2i,s2i,in2i])
    query, slots, intent =  map(train_ds.get, ['query', 'slot_labels', 'intent_labels'])

    if add_start_end_token:
        i2s[178] = 'BOS'
        i2s[179] = 'EOS'
        s2i['BOS'] = 178
        s2i['EOS'] = 179

    input_tensor = []
    target_tensor = []
    query_data = []
    intent_data = []
    slot_data = []
    to_show = np.random.randint(0, len(query)-1, 5)
    for i in range(len(query)):
        input_tensor.append(query[i])
        slot_text = []
        slot_vector = []
        for j in range(len(query[i])):
            slot_text.append(i2s[slots[i][j]])
            slot_vector.append(int(slots[i][j]))
        if add_start_end_token:
            slot_text[0] = 'BOS'
            slot_vector[0] = 178
            slot_text[-1] = 'EOS'
            slot_vector[-1]= 179
        target_tensor.append(slot_vector)
        q = ' '.join(map(i2t.get, query[i]))
        query_data.append(q.replace('BOS', '').replace('EOS',''))
        intent_data.append(i2in[intent[i][0]])
        slot = ' '.join(slot_text)
        slot_data.append(slot[1:-1])
        if i in to_show and verbose:
            print('Query text:', q)
            print('Query vector: ', query[i])
            print('Intent label: ', i2in[intent[i][0]])
            print('Slot text: ', slot)
            print('Slot vector: ', slot_vector)
            print('*'*74)

    query_data = np.array(query_data)
    intent_data = np.array(intent_data)
    slot_data = np.array(slot_data)
    # one intent id per query (the intent is repeated for every token, so take the first one)
    intent_data_label = np.array([int(np.ravel(x)[0]) for x in intent])

    return t2i, s2i, in2i, i2t, i2s, i2in, input_tensor, target_tensor, query_data, intent_data, intent_data_label, slot_data
```

Cell ini berisi fungsi load_ds() dan load_atis() untuk membaca serta mengolah dataset ATIS. Fungsi load_ds() membuka file Pickle dan mengambil data beserta informasi kosakata, label *intent*, dan *slot*. Sementara itu, fungsi load_atis() mengubah data berbentuk angka menjadi teks agar lebih mudah dipahami dan digunakan. Hasilnya berupa kumpulan kalimat, label *intent*, label *slot*, serta informasi lain yang diperlukan untuk proses klasifikasi.

Cell 4 – Memuat Dataset Training dan Testing

```
# load ATIS training dataset
t2i_train, s2i_train, in2i_train, i2t_train, i2s_train, i2in_train, \
input_tensor_train, target_tensor_train, \
query_data_train, intent_data_train, intent_data_label_train, slot_data_train = load_atis('atis.train.pkl')

# load ATIS testing dataset
t2i_test, s2i_test, in2i_test, i2t_test, i2s_test, i2in_test, \
input_tensor_test, target_tensor_test, \
query_data_test, intent_data_test, intent_data_label_test, slot_data_test = load_atis('atis.test.pkl')
```

```
Done  loading:  ./atis.train.pkl
     samples: 4978
   vocab_size:  943
   slot count:  129
 intent count:   26
Query text: BOS can you give me information on transportation from the airport in philadelphia to downtown philadelphia EOS
Query vector:  [178 282 938 449 581 492 654 866 444 827 203 482 678 851 380 678 179]
Intent label:  ground_service
Slot text:  O O O O O O O O O O O B-fromloc.city_name O O B-toloc.city_name O
Slot vector:  [128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 128, 48, 128, 128, 78, 128]
********************************************************************************
Query text: BOS flight from denver to boston EOS
Query vector:  [178 428 444 351 851 266 179]
Intent label:  flight
Slot text:  O O O B-fromloc.city_name O B-toloc.city_name O
Slot vector:  [128, 128, 128, 48, 128, 78, 128]
********************************************************************************
Query text: BOS please give me flight schedules from pittsburgh to philadelphia EOS
Query vector:  [178 688 449 581 428 747 444 682 851 678 179]
Intent label:  flight
Slot text:  O O O O B-flight_time I-flight_time O B-fromloc.city_name O B-toloc.city_name O
Slot vector:  [128, 128, 128, 128, 45, 108, 128, 48, 128, 78, 128]
********************************************************************************
Query text: BOS list all takeoffs and landings at general mitchell international EOS
Query vector:  [178 549 207 813 215 524 240 445 600 496 179]
Intent label:  flight
Slot text:  O O O O O O O B-airport_name I-airport_name I-airport_name O
Slot vector:  [128, 128, 128, 128, 128, 128, 128, 4, 84, 84, 128]
********************************************************************************
Query text: BOS show me round trip first class tickets from new york to miami EOS
Query vector:  [178 770 581 730 870 425 302 848 444 619 937 851 589 179]
Intent label:  airfare
Slot text:  O O O B-round_trip I-round_trip B-class_type I-class_type O O B-fromloc.city_name I-fromloc.city_name O B-toloc.city_name
Slot vector:  [128, 128, 128, 66, 119, 18, 92, 128, 128, 48, 110, 128, 78, 128]
********************************************************************************
********************************************************************************
Done  loading:  ./atis.test.pkl
     samples:  893
   vocab_size:  943
   slot count:  129
 intent count:   26
Query text: BOS what airlines off from love field between 6 and 10 am on june sixth EOS
Query vector:  [178 916 200 647 444 565 418 259 130 215  10 210 654 508 775 179]
Intent label:  airline
Slot text:  O O O O O B-fromloc.airport_name I-fromloc.airport_name O B-depart_time.start_time O B-depart_time.end_time I-depart_time.end
Slot vector:  [128, 128, 128, 128, 128, 47, 109, 128, 34, 128, 31, 97, 128, 28, 27, 128]
********************************************************************************
Query text: BOS st. paul to kansas city friday night EOS
Query vector:  [178 789 673 851 511 301 441 622 179]
Intent label:  flight
Slot text:  O B-fromloc.city_name I-fromloc.city_name O B-toloc.city_name I-toloc.city_name B-depart_date.day_name B-depart_time.period_o
Slot vector:  [128, 48, 110, 128, 78, 125, 26, 33, 128]
********************************************************************************
Query text: BOS what are restrictions ap 57 EOS
Query vector:  [178 916 228 725 221 129 179]
Intent label:  abbreviation
Slot text:  O O O O B-restriction_code I-restriction_code O
Slot vector:  [128, 128, 128, 128, 58, 115, 128]
********************************************************************************
Query text: BOS how about a flight leaving tuesday night from st. louis to burbank EOS
Query vector:  [178 477 185 180 428 539 874 622 444 789 564 851 272 179]
Intent label:  flight
Slot text:  O O O O O O B-depart_date.day_name B-depart_time.period_of_day O B-fromloc.city_name I-fromloc.city_name O B-toloc.city_name
Slot vector:  [128, 128, 128, 128, 128, 128, 26, 33, 128, 48, 110, 128, 78, 128]
********************************************************************************
Query text: BOS like to book a flight from burbank to milwaukee EOS
Query vector:  [178 545 851 264 180 428 444 272 851 595 179]
Intent label:  flight
Slot text:  O O O O O O O B-fromloc.city_name O B-toloc.city_name O
Slot vector:  [128, 128, 128, 128, 128, 128, 128, 48, 128, 78, 128]
********************************************************************************
```

Cell ini digunakan untuk memuat dataset ATIS yang akan digunakan dalam praktikum. Data pelatihan dimuat dari file atis.train.pkl, sedangkan data pengujian diambil dari file atis.test.pkl. Hasil pemuatan disimpan dalam beberapa variabel yang berisi kalimat, label *intent*, label *slot*, dan informasi kosakata. Data tersebut kemudian digunakan untuk tahap pemeriksaan dan pengolahan sebelum model BERT dilatih.

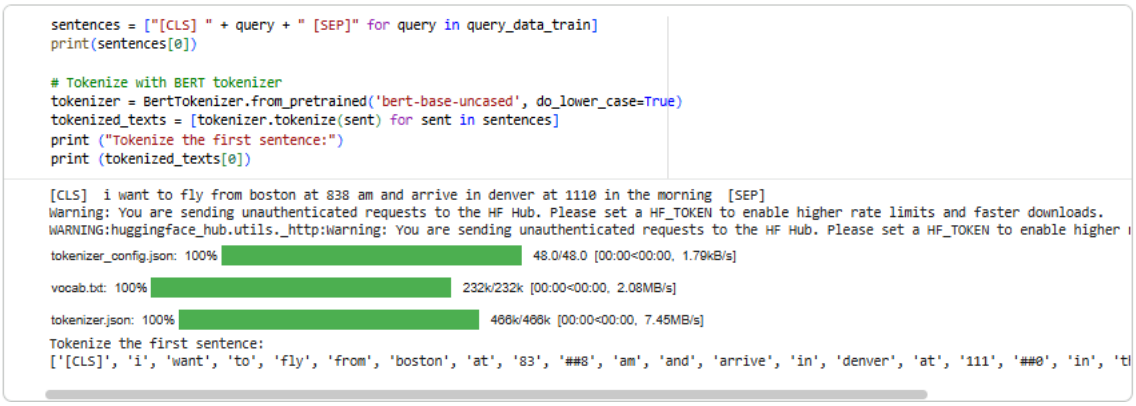
Cell 5 – Memeriksa Dataset

```
print("Number of training sentences :", len(query_data_train))
print("Number of unique intents     :", len(set(intent_data_train)))
query_data_train
```

```
Number of training sentences : 4978
Number of unique intents     : 22
array([' i want to fly from boston at 838 am and arrive in denver at 1110 in the morning ',
       ' what flights are available from pittsburgh to baltimore on thursday morning ',
       ' what is the arrival time in san francisco for the 755 am flight leaving washington ',
       ..., ' which airlines fly into and out of denver ',
       ' does continental fly from boston to san francisco with a stop in denver ',
       ' is there a delta flight from denver to san francisco '],
      dtype='<U261')
```

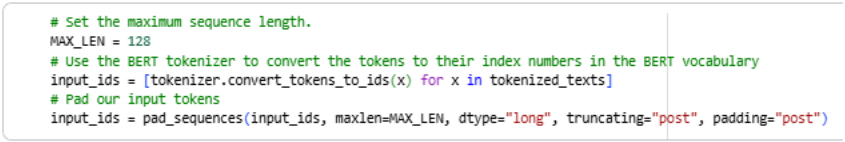
Cell ini digunakan untuk menampilkan jumlah kalimat dalam dataset pelatihan dan jumlah kategori *intent* yang berbeda. Fungsi len() menghitung jumlah kalimat, sedangkan set() membantu mengetahui banyaknya kategori *intent* unik. Baris query_data_train juga menampilkan data kalimat pelatihan yang telah dimuat. Pemeriksaan ini membantu memastikan bahwa dataset berhasil dibaca dan memberikan gambaran awal mengenai data yang digunakan.

Cell 6 – Tokenisasi Kalimat



Cell ini menyiapkan kalimat dengan menambahkan token [CLS] di awal dan [SEP] di akhir kalimat sesuai format masukan BERT. Selanjutnya, tokenizer bert-base-uncased digunakan untuk memecah kalimat menjadi bagian-bagian token yang dapat dikenali model. Tokenisasi membantu BERT memahami teks berdasarkan kosakata yang telah dipelajari sebelumnya. Hasil tokenisasi dari kalimat pertama ditampilkan agar proses pengolahan teks dapat diamati secara langsung.

Cell 7 – Mengubah Token Menjadi Angka dan Padding



Cell ini mengubah token hasil tokenisasi menjadi ID angka menggunakan kosakata BERT. Variabel MAX_LEN = 128 menetapkan panjang maksimum setiap urutan menjadi 128 token. Fungsi pad_sequences() kemudian menambahkan angka nol pada urutan yang lebih pendek agar seluruh data memiliki panjang yang sama. Jika panjang urutan melebihi batas tersebut, token yang berada di bagian akhir akan dipotong.

Cell 8 – Membuat Attention Masks



Cell ini membuat *attention masks* untuk membedakan token asli dari angka nol yang ditambahkan melalui proses *padding*. Setiap nilai pada input_ids yang lebih besar dari nol akan diberi nilai 1, sedangkan nilai nol akan diberi nilai 0. Penanda tersebut membantu BERT mengetahui bagian mana dari urutan yang perlu diperhatikan saat memproses kalimat. Hasil *attention masks* nantinya digunakan bersama ID token sebagai masukan model.

Cell 9 –Mengubah Label Menjadi Klasifikasi Biner

```
# Reducing this into a binary classification problem: intent id 14 -> 1, all other intents -> 0
print("Intent id 14 is:", i2in_train[14])
intent_data_label_binary = np.where(intent_data_label_train == 14, 1, 0)
print("Class counts (label 0, label 1):", np.bincount(intent_data_label_binary))
```

```
Intent id 14 is: flight
Class counts (label 0, label 1): [1312 3666]
```

Cell ini mengubah label *intent* yang awalnya terdiri dari banyak kategori menjadi dua kelas. Berdasarkan kode, *intent* dengan ID 14 diberi label 1, sedangkan semua *intent* lainnya diberi label 0. Fungsi np.where() digunakan untuk mengubah label tersebut sesuai dengan kondisi yang telah ditentukan. Selanjutnya, np.bincount() menghitung jumlah data pada masing-masing kelas untuk melihat distribusi label.

Cell 10 – Membagi Data Training dan Validation

```
# Use train_test_split to split our data into train and validation sets for training
train_inputs, validation_inputs, train_labels, validation_labels = train_test_split(input_ids, intent_data_label_binary,
                                                            random_state=2018, test_size=0.1)
train_masks, validation_masks, _, _ = train_test_split(attention_masks, input_ids,
                                             random_state=2018, test_size=0.1)

# Convert all of our data into torch tensors, the required datatype for our model
train_inputs = torch.tensor(train_inputs)
validation_inputs = torch.tensor(validation_inputs)
train_labels = torch.tensor(train_labels)
validation_labels = torch.tensor(validation_labels)
train_masks = torch.tensor(train_masks)
validation_masks = torch.tensor(validation_masks)

# Select a batch size for training.
batch_size = 32

# Create an iterator of our data with torch DataLoader
train_data = TensorDataset(train_inputs, train_masks, train_labels)
train_sampler = RandomSampler(train_data)
train_dataloader = DataLoader(train_data, sampler=train_sampler, batch_size=batch_size)
validation_data = TensorDataset(validation_inputs, validation_masks, validation_labels)
validation_sampler = SequentialSampler(validation_data)
validation_dataloader = DataLoader(validation_data, sampler=validation_sampler, batch_size=batch_size)
```

Cell ini membagi data menjadi data pelatihan dan validasi dengan proporsi 90% dan 10%. Parameter random_state=2018 digunakan agar pembagian data dapat diulang dengan hasil yang sama, sedangkan test_size=0.1 menentukan proporsi data validasi. Selanjutnya, data kalimat, label, dan *attention masks* diubah menjadi *tensor* PyTorch agar dapat diproses oleh model. DataLoader kemudian mengelompokkan data dalam *batch* berukuran 32, dengan pengacakan untuk data pelatihan dan urutan berurutan untuk data validasi.

Cell 11 – Memuat Model BERT

```
model = BertForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)
model.to(device)
```

```
config.json: 100% 570/570 [00:00<00:00, 57.6kB/s]
model.safetensors: reconstructing file: 100% 440MB / 440MB, 40.4MB/s
model.safetensors: downloading bytes: 415MB, 37.4MB/s
Loading weights: 100% 199/199 [00:00<00:00, 5201.74it/s]
[transformers] BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                        | Status     |
-------------------------------------------+------------+
cls.seq_relationship.weight                | UNEXPECTED |
cls.seq_relationship.bias                  | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias   | UNEXPECTED |
cls.predictions.transform.dense.bias       | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
cls.predictions.transform.dense.weight     | UNEXPECTED |
cls.predictions.bias                       | UNEXPECTED |
classifier.bias                            | MISSING    |
classifier.weight                          | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING:  those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
BertForSequenceClassification(
  (bert): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
```

Cell ini digunakan untuk memuat model BertForSequenceClassification dengan model dasar bert-base-uncased. Parameter num_labels=2 menunjukkan bahwa model disiapkan untuk mengklasifikasikan teks ke dalam dua kelas. Model BERT yang telah dilatih sebelumnya akan disesuaikan dengan dataset ATIS melalui proses *fine-tuning*. Perintah model.to(device) digunakan untuk menempatkan model pada GPU jika tersedia atau CPU jika GPU tidak tersedia.

Cell 12 – Membuat Grafik Training Loss

```
# BERT fine-tuning parameters
param_optimizer = list(model.named_parameters())
no_decay = ['bias', 'LayerNorm.weight']
optimizer_grouped_parameters = [
    {'params': [p for n, p in param_optimizer if not any(nd in n for nd in no_decay)],
     'weight_decay': 0.01},
    {'params': [p for n, p in param_optimizer if any(nd in n for nd in no_decay)],
     'weight_decay': 0.0}
]
int: epochs
4
ining epochs
epochs = 4

optimizer = torch.optim.AdamW(optimizer_grouped_parameters, lr=2e-5)
total_steps = len(train_dataloader) * epochs
scheduler = get_linear_schedule_with_warmup(
    optimizer, num_warmup_steps=int(0.1 * total_steps), num_training_steps=total_steps)

# Function to calculate the accuracy of our predictions vs labels
def flat_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=1).flatten()
    labels_flat = labels.flatten()
    return np.sum(pred_flat == labels_flat) / len(labels_flat)

# Store our loss for plotting
train_loss_set = []

# BERT training loop
for _ in trange(epochs, desc="Epoch"):

    ## TRAINING

    # Set our model to training mode
    model.train()
    # Tracking variables
    tr_loss = 0
    nb_tr_examples, nb_tr_steps = 0, 0
    # Train the data for one epoch
    for step, batch in enumerate(train_dataloader):
        # Add batch to GPU
        batch = tuple(t.to(device) for t in batch)
        # Unpack the inputs from our dataloader
        b_input_ids, b_input_mask, b_labels = batch
        # Clear out the gradients (by default they accumulate)
        optimizer.zero_grad()
        # Forward pass (the model now returns an output object, the loss is outputs.loss)
        outputs = model(b_input_ids, token_type_ids=None, attention_mask=b_input_mask, labels=b_labels)
        loss = outputs.loss
        train_loss_set.append(loss.item())
        # Backward pass
        loss.backward()
```

are complete

OK

```
        # Clip gradients to avoid exploding gradients
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        # Update parameters and learning rate
        optimizer.step()
        scheduler.step()
        # Update tracking variables
        tr_loss += loss.item()
        nb_tr_examples += b_input_ids.size(0)
        nb_tr_steps += 1
    print("Train loss: {}".format(tr_loss/nb_tr_steps))

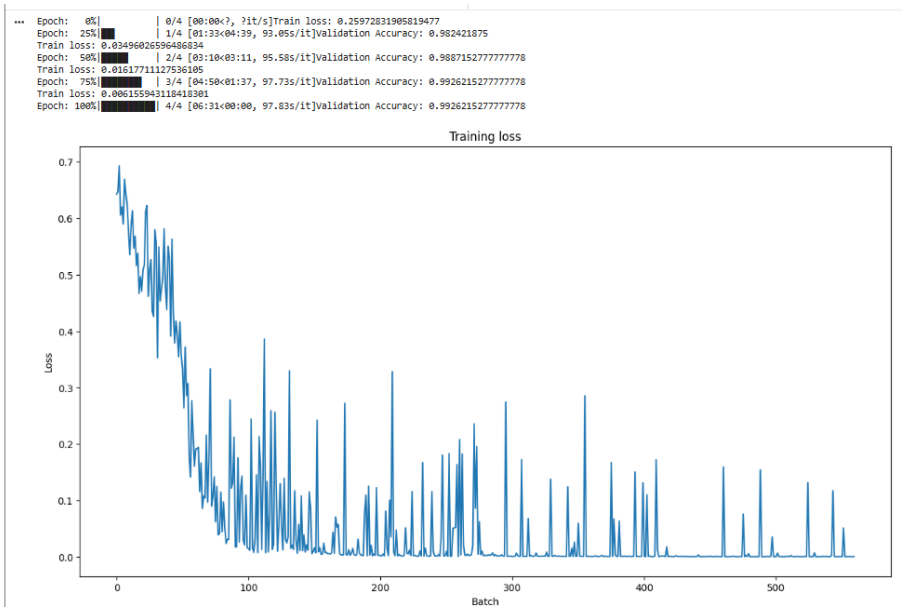
    ## VALIDATION

    # Put model in evaluation mode
    model.eval()
    # Tracking variables
    eval_loss, eval_accuracy = 0, 0
    nb_eval_steps, nb_eval_examples = 0, 0
    # Evaluate data for one epoch
    for batch in validation_dataloader:
        # Add batch to GPU
        batch = tuple(t.to(device) for t in batch)
        # Unpack the inputs from our dataloader
        b_input_ids, b_input_mask, b_labels = batch
        # Telling the model not to compute or store gradients, saving memory and speeding up validation
        with torch.no_grad():
            # Forward pass, calculate logit predictions
            outputs = model(b_input_ids, token_type_ids=None, attention_mask=b_input_mask)
        # Move logits and labels to CPU
        logits = outputs.logits.detach().cpu().numpy()
        label_ids = b_labels.to('cpu').numpy()
        tmp_eval_accuracy = flat_accuracy(logits, label_ids)
        eval_accuracy += tmp_eval_accuracy
        nb_eval_steps += 1
    print("Validation Accuracy: {}".format(eval_accuracy/nb_eval_steps))

# plot training performance
plt.figure(figsize=(15,8))
plt.title("Training loss")
plt.xlabel("Batch")
plt.ylabel("Loss")
plt.plot(train_loss_set)
plt.show()
```

are complete

OK



Cell ini menjalankan proses *fine-tuning* model BERT selama empat *epoch* menggunakan optimizer AdamW dengan *learning rate* sebesar 2e-5. Pada setiap *epoch*, program menampilkan nilai *training loss* untuk melihat tingkat kesalahan model selama proses pelatihan serta *validation accuracy* untuk mengetahui ketepatan prediksi pada data validasi. Grafik tersebut menunjukkan perubahan nilai training loss selama proses pelatihan model BERT. Pada awal pelatihan, nilai loss berada di kisaran 0,6–0,7, kemudian cenderung menurun hingga mendekati 0. Hal ini menunjukkan bahwa model semakin baik dalam mempelajari pola dari data pelatihan. Meskipun terdapat beberapa lonjakan nilai loss pada batch tertentu, secara keseluruhan grafik memperlihatkan tren penurunan yang cukup jelas.

04_CRF_SNIPS
Link Google Colab:
https://colab.research.google.com/drive/1pZH2XlnN8QujdffsMH-DKTGpfJH1GWxq?usp=sharing

Cell 1 – Instalasi Library

```
#Instalasi Library
!pip install sklearn-crfsuite nltk
```

```
Collecting sklearn-crfsuite
  Downloading sklearn_crfsuite-0.5.0-py2.py3-none-any.whl.metadata (4.9 kB)
Requirement already satisfied: nltk in /usr/local/lib/python3.13/dist-packages (3.9.1)
Collecting python-crfsuite>=0.9.7 (from sklearn-crfsuite)
  Downloading python_crfsuite-0.9.12-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (4.3 kB)
Requirement already satisfied: scikit-learn>=0.24.0 in /usr/local/lib/python3.13/dist-packages (from sklearn-crfsuite) (1.6.1)
Requirement already satisfied: tabulate>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from sklearn-crfsuite) (0.9.0)
Requirement already satisfied: tqdm>=2.0 in /usr/local/lib/python3.13/dist-packages (from sklearn-crfsuite) (4.67.3)
Requirement already satisfied: click in /usr/local/lib/python3.13/dist-packages (from nltk) (8.5.0)
Requirement already satisfied: joblib in /usr/local/lib/python3.13/dist-packages (from nltk) (1.6.0)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.13/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: numpy>=1.19.5 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (2.1.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=0.24.0->sklearn-crfsuite) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib->nltk) (3.1.2)
Downloading sklearn_crfsuite-0.5.0-py2.py3-none-any.whl (10 kB)
Downloading python_crfsuite-0.9.12-cp313-cp313-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (1.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 34.3 MB/s eta 0:00:00
Installing collected packages: python-crfsuite, sklearn-crfsuite
Successfully installed python-crfsuite-0.9.12 sklearn-crfsuite-0.5.0
```

Cell ini menginstal library sklearn-crfsuite digunakan untuk membangun model CRF, sedangkan nltk membantu proses pengolahan teks dan pengenalan kelas kata. Proses instalasi juga memeriksa dependensi yang dibutuhkan oleh kedua library tersebut. Berdasarkan output yang muncul, instalasi berhasil diselesaikan tanpa kendala. Dengan demikian, library sudah siap digunakan pada tahap berikutnya.

Cell 2 – Import Library dan Download Data NLTK

```
#Import Library dan Download Data NLTK
import os
import json
import numpy as np
import nltk
from sklearn_crfsuite import CRF, metrics
from sklearn.metrics import confusion_matrix

# Mengunduh modul NLTK yang dibutuhkan untuk ekstraksi fitur POS Tag
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng') # Antisipasi untuk versi NLTK terbaru
from nltk.tag import pos_tag
```

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
```

Cell ini mengmpor beberapa library yang dibutuhkan, seperti os, json, numpy, nltk, dan sklearn-crfsuite. Setiap library memiliki fungsi yang berbeda, mulai dari mengelola file JSON, melakukan perhitungan numerik, hingga membangun dan mengevaluasi model CRF. Selanjutnya, kode mengunduh modul POS Tagger dari NLTK yang digunakan untuk mengenali kelas kata, seperti kata benda dan kata kerja. Informasi kelas kata tersebut nantinya membantu proses ekstraksi fitur dalam teks. Berdasarkan output, kedua modul NLTK berhasil diunduh dan diekstrak.

Cell 3 – Mengunggah dan Membaca File JSON

```
#Mengunggah File JSON
from google.colab import files
print("Silakan upload file train_PlayMusic_full.json dan validate_PlayMusic.json")
uploaded = files.upload()

# Membaca data JSON
try:
    with open("train_PlayMusic_full.json", encoding="iso-8859-2") as f:
        train_file = json.load(f)
    with open("validate_PlayMusic.json", encoding="iso-8859-2") as f:
        test_file = json.load(f)

    train_datafile = [i["data"] for i in train_file["PlayMusic"]]
    test_datafile = [i["data"] for i in test_file["PlayMusic"]]
    print("File JSON berhasil dimuat!")
except FileNotFoundError:
    print("Error: File JSON tidak ditemukan. Pastikan Anda telah mengunggah kedua file tersebut.")
```

Cell ini digunakan untuk mengupload 2 file JSON, yaitu train_PlayMusic_full.json dan validate_PlayMusic.json. File pertama digunakan sebagai data latih, sedangkan file kedua digunakan sebagai data validasi untuk menguji model. Setelah diunggah, program membaca isi kedua file menggunakan format encoding iso-8859-2 agar data dapat diproses sesuai pengaturan yang digunakan. Selanjutnya, bagian data dengan kunci PlayMusic dan data diambil untuk dipersiapkan ke tahap berikutnya. Berdasarkan output yang ditampilkan, kedua file berhasil dimuat tanpa kendala.

Cell 4 – Konversi Data dan Ekstrasi Entitas

```
#Konversi Data dan Ekstraksi Entitas
def convert_data(data):
    output = []
    # Loop pertama: iterasi untuk setiap kalimat utuh
    for item in data:
        sent = []
        pos = []

        # Loop kedua: iterasi untuk setiap potongan kata/entitas di dalam kalimat
        for phrase in item:
            words = phrase["text"].strip().split(" ")

            while "" in words:
                words.remove("")

            if "entity" in phrase.keys():
                label = phrase["entity"]
                labels = [f"{label}-{i+1}" for i in range(len(words))]
            else:
                labels = ["O"] * len(words)

            sent.extend(words)
            pos.extend(labels)

        output.append([sent, pos])

        # PERINTAH INI MENGHASILKAN OUTPUT PANJANG SESUAI PERMINTAAN ANDA
        print(sent)

    return output

print("Memproses train_data...")
train_data = convert_data(train_datafile)

print("\nMemproses test_data...")
test_data = convert_data(test_datafile)
```

```
['Find', 'a', 'top', 'fifty', 'zahir', 'howaida', 'concerto', 'from', 'the', 'seventies']
['Play', 'the', 'newest', 'twenties', 'melody', '.']
['I', 'want', 'to', 'hear', 'Box', 'Of', 'Rain', 'by', 'Skeets', 'Mcdonald']
['Please', 'play', 'Casino', 'Boogie']
['I', 'want', 'to', 'hear', 'a', 'full', 'album', 'on', 'youtube', 'by', 'Jem', 'Godfrey']
['Play', 'an', 'Asha', 'Bhosle', 'song', 'from', 'around', '1964']
['Play', 'music', 'from', 'the', 'twenties', '.']
['Play', 'something', 'by', 'Louisiana', 'Blues']
['Play', 'new', 'Ian', 'Mclagan']
['Play', 'music', 'on', 'Itunes']
['I', 'want', 'to', 'hear', 'Ding', 'Dong', 'Bell', 'on', 'groove', 'shark']
['Play', 'fifties', 'music', 'on', 'Slacker', '.']
['Play', 'Before', 'I', 'Grew', 'Up', 'To', 'Love', 'You', 'by', 'Wafah', 'Dufour']
['Play', 'a', 'Jim', 'Mullen', 'ep', 'from', 'the', 'sixties']
['Play', 'some', 'good', 'music', 'from', '2012', '.']
['Play', 'the', 'greatest', 'soundtrack', 'by', 'Nhat', 'Son', 'on', 'Last', 'Fm', '.']
['Play', 'the', 'top', 'hits', 'of', '2016']
['Play', 'Fight', 'On,', 'State', '.']
['Use', 'Pandora', 'to', 'play', 'music', 'from', 'Maki', 'Onaga', 'from', 'the', 'album', 'made'
['Play', 'the', 'top', 'five', 'Cemil', 'Bey', 'songs']
['Play', 'opera', 'music', 'on', 'google', 'music']
['play', 'music', 'by', 'Francis', 'Healy']
['play', 'a', 'track', 'from', 'the', 'thirties']
['Play', 'music', 'on', 'Spotify', '.']
['play', 'newest', 'soundtrack', 'from', '2008']
['play', 'Itunes', 'songs', 'by', 'Gerard', 'Way']
['Play', 'Cherry', 'Pink', 'by', 'Trapp', 'Mendoza', 'on', 'Iheart']
['Play', 'Rei', 'Momo', 'newest', 'album']
['I', 'want', 'to', 'hear', 'a', 'track', 'by', 'Ryohei', 'Yamamoto', 'on', 'Vimeo']
['Start', 'up', 'my', 'nostalgic', 'playlist.']
['play', 'some', 'papoose', 'from', '2003', 'on', 'youtube']
['Use', 'Deezer', 'to', 'play', 'music', 'by', 'Kimberley', 'Walsh']
['Play', 'Punk', 'Rock', 'music']
['Play', 'something', 'by', 'Louis', 'Nelson', 'Delisle', '.']
['play', '1962', 'music', 'on', 'Netflix']
['Play', 'a', 'song', 'by', 'David', 'Silveria', 'from', '2002', 'on', 'Zvooq', '.']
['play', 'the', '2014', 'album', 'from', 'la', 'lupe']
['Play', 'Revival', 'music.']
['I', 'want', 'to', 'hear', 'opera', 'on', 'Netflix']
['play', 'the', 'song', 'Si', 'No', 'Te', 'Hubiera', 'Conocido', 'by', 'Haidar', 'Salim']
```

Cell ini menggunakan fungsi convert_data() untuk mengubah data JSON menjadi kumpulan kata beserta label entitasnya. Setiap kalimat diproses dengan memisahkan teks menjadi kata-kata berdasarkan spasi. Kata yang memiliki entitas akan diberi label sesuai jenis entitasnya, sedangkan kata

yang tidak memiliki entitas diberi label O. Hasil pemrosesan disusun menjadi pasangan data berupa daftar kata dan daftar label yang sesuai. Proses ini dilakukan pada data latih dan data validasi agar keduanya memiliki format yang dapat digunakan oleh model CRF.

Cell 5 – Download dan Unzip GloVe

```
#Download dan Unzip GloVe
!wget -nc http://nlp.stanford.edu/data/glove.6B.zip
!unzip -n glove.6B.zip -d glove.6B

# Memuat vektor kata ke dalam memori
print('\nPreparing embedding matrix...')
embeddings_index = {}
with open('glove.6B/glove.6B.100d.txt', encoding="utf-8") as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embeddings_index[word] = coefs

print(f'Found {len(embeddings_index)} word vectors in Glove embeddings.')

EMBEDDING_DIM = 100
def get_embeddings(word):
    embedding_vector = embeddings_index.get(word)
    if embedding_vector is None:
        embedding_vector = np.zeros(shape=(EMBEDDING_DIM,))
    return embedding_vector
```

```
--2026-10-11 10:11:15--  http://nlp.stanford.edu/data/glove.6B.zip
Resolving nlp.stanford.edu (nlp.stanford.edu)... 171.64.67.140
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-10-11 10:11:15--  https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-10-11 10:11:16--  https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
```

Cell ini digunakan untuk mendownload dan ekstraksi GloVe (Global Vectors for Word Representation) yang digunakan untuk merepresentasikan kata dalam bentuk vektor numerik. File GloVe yang digunakan memiliki ukuran vektor 100 dimensi untuk setiap kata. Selanjutnya, setiap kata dan vektornya disimpan dalam variabel embeddings_index agar dapat diakses saat proses ekstraksi fitur. Fungsi get_embeddings() digunakan untuk mengambil vektor dari kata yang tersedia di dalam data GloVe. Jika kata tidak ditemukan, program akan memberikan vektor bernilai nol sebagai pengganti.

Cell 6 – Fungsi Ekstraksi Fitur

```
#Fungsi Ekstraksi Fitur
def sent2feats(sentence):
    feats = []
    sen_tags = pos_tag(sentence) # Format spesifik untuk POS Tagger ini

    for i in range(len(sentence)):
        word = sentence[i]
        wordfeats = {}

        # Fitur kata saat ini, sebelumnya, dan sesudahnya
        wordfeats['word'] = word

        if i == 0:
            wordfeats["prevWord"] = wordfeats["prevSecondWord"] = "<S>"
        elif i == 1:
            wordfeats["prevWord"] = sentence[0]
            wordfeats["prevSecondWord"] = "<S>"
        else:
            wordfeats["prevWord"] = sentence[i-1]
            wordfeats["prevSecondWord"] = sentence[i-2]

        if i == len(sentence) - 2:
            wordfeats["nextWord"] = sentence[i+1]
            wordfeats["nextNextWord"] = "</S>"
        elif i == len(sentence) - 1:
            wordfeats["nextWord"] = "</S>"
            wordfeats["nextNextWord"] = "</S>"
        else:
            wordfeats["nextWord"] = sentence[i+1]
            wordfeats["nextNextWord"] = sentence[i+2]
```

```
            # Fitur POS Tag
            wordfeats['tag'] = sen_tags[i][1]

            if i == 0:
                wordfeats["prevTag"] = wordfeats["prevSecondTag"] = "<S>"
            elif i == 1:
                wordfeats["prevTag"] = sen_tags[0][1]
                wordfeats["prevSecondTag"] = "<S>"
            else:
                wordfeats["prevTag"] = sen_tags[i-1][1]
                wordfeats["prevSecondTag"] = sen_tags[i-2][1]

            if i == len(sentence) - 2:
                wordfeats["nextTag"] = sen_tags[i+1][1]
                wordfeats["nextNextTag"] = "</S>"
            elif i == len(sentence) - 1:
                wordfeats["nextTag"] = "</S>"
                wordfeats["nextNextTag"] = "</S>"
            else:
                wordfeats["nextTag"] = sen_tags[i+1][1]
                wordfeats["nextNextTag"] = sen_tags[i+2][1]

            # Menambahkan vektor kata GloVe ke dalam fitur CRF
            vector = get_embeddings(word)
            for iv, value in enumerate(vector):
                wordfeats[f'v{iv}'] = value

            feats.append(wordfeats)
        return feats
```

```
def get_feats_conll(conll_data):
    feats = []
    labels = []
    for sentence in conll_data:
        feats.append(sent2feats(sentence[0]))
        labels.append(sentence[1])
    return feats, labels

# Menjalankan ekstraksi fitur pada dataset
print("Mengekstrak fitur, mohon tunggu...")
train_feats, train_labels = get_feats_conll(train_data)
test_feats, test_labels = get_feats_conll(test_data)
print("Ekstraksi selesai.")
```

```
Mengekstrak fitur, mohon tunggu...
Ekstraksi selesai.
```

Cell ini menggunakan fungsi sent2feats() untuk mengambil fitur yang dibutuhkan dalam proses pengenalan entitas menggunakan model CRF. Fitur yang digunakan mencakup kata saat ini, kata sebelum dan sesudahnya, serta hasil POS Tagging untuk mengetahui kelas kata. Selain itu, vektor kata dari GloVe ditambahkan sebagai fitur numerik yang membantu merepresentasikan karakteristik kata. Selanjutnya, fungsi get_feats_conll() digunakan untuk memproses seluruh kalimat dan memisahkan fitur dengan label entitasnya. Berdasarkan output, proses ekstraksi fitur pada data latih dan data validasi berhasil diselesaikan.

Cell 7 – Fungsi Confusion Matrix

```
#Custom Confusion Matrix Print
def print_cm(cm, labels):
    print("\n")
    print("=== Confusion Matrix ===")
    columnwidth = max([len(x) for x in labels] + [5])
    empty_cell = " " * columnwidth

    # Print header
    print("    " + empty_cell, end=" ")
    for label in labels:
        print("%{0}s".format(columnwidth) % label, end=" ")
    print()

    # Print baris
    for i, label1 in enumerate(labels):
        print("    %{0}s".format(columnwidth) % label1, end=" ")
        sum_row = 0
        for j in range(len(labels)):
            cell = "%{0}.0f".format(columnwidth) % cm[i, j]
            sum_row += int(cell)
            print(cell, end=" ")
        print(sum_row) # Print total instance per kategori di akhir

def get_confusion_matrix(y_true, y_pred, labels):
    trues, preds = [], []
    for yseq_true, yseq_pred in zip(y_true, y_pred):
        trues.extend(yseq_true)
        preds.extend(yseq_pred)
    print_cm(confusion_matrix(trues, preds, labels=labels), labels)
```

Cell ini menggunakan fungsi print_cm() untuk menampilkan hasil Confusion Matrix dalam format tabel agar lebih mudah dibaca. Tabel tersebut memperlihatkan perbandingan antara label entitas sebenarnya dan label yang diprediksi oleh model. Fungsi get_confusion_matrix() digunakan untuk menggabungkan label dari seluruh kalimat sebelum menghitung matriks evaluasi. Hasilnya menunjukkan jumlah prediksi yang sesuai maupun yang berbeda dari label sebenarnya pada setiap kategori. Informasi ini dapat digunakan untuk mengetahui kategori entitas yang berhasil dikenali model dan kategori yang masih sering mengalami kesalahan prediksi.

Cell 8 – Training & Evaluation CRF

```python
#Training & Evaluation CRF
def train_seq(X_train, Y_train, X_dev, Y_dev):
    print("Memulai pelatihan model CRF...")
    crf = CRF(algorithm='lbfgs', c1=0.1, c2=10, max_iterations=50, all_possible_transitions=True)

    # Melatih model
    crf.fit(X_train, Y_train)
    labels = list(crf.classes_)

    # Memprediksi data uji
    y_pred = crf.predict(X_dev)

    # Evaluasi
    sorted_labels = sorted(labels, key=lambda name: (name[1:], name[0]))

    print("\n--- F1 Score ---")
    print(metrics.flat_f1_score(Y_dev, y_pred, average='weighted', labels=labels))

    print("\n--- Classification Report ---")
    print(metrics.flat_classification_report(Y_dev, y_pred, labels=sorted_labels, digits=3))

    get_confusion_matrix(Y_dev, y_pred, labels=sorted_labels)

# Jalankan pelatihan
train_seq(train_feats, train_labels, test_feats, test_labels)
```

```
Memulai pelatihan model CRF...

--- F1 Score ---
0.8593665180268206

--- Classification Report ---
              precision    recall  f1-score   support

           O      0.979     0.981     0.980       418
      year-1      0.960     0.960     0.960        25
     genre-1      1.000     0.333     0.500         3
     genre-2      1.000     0.333     0.500         3
     genre-3      0.000     0.000     0.000         0
     genre-4      0.000     0.000     0.000         0
     genre-5      0.000     0.000     0.000         0
     genre-6      0.000     0.000     0.000         0
   service-1      0.837     0.923     0.878        39
   service-2      1.000     0.818     0.900        11
  playlist-1      1.000     0.333     0.500         9
  playlist-2      0.333     0.167     0.222         6
  playlist-3      0.000     0.000     0.000         4
  playlist-4      0.000     0.000     0.000         4
  playlist-5      0.000     0.000     0.000         2
  playlist-6      0.000     0.000     0.000         0
     album-1      0.000     0.000     0.000        13
     album-2      0.000     0.000     0.000        12
     album-3      0.000     0.000     0.000         7
     album-4      0.000     0.000     0.000         2
     album-5      0.000     0.000     0.000         1
     album-6      0.000     0.000     0.000         0
     album-7      0.000     0.000     0.000         0
     album-8      0.000     0.000     0.000         0
      sort-1      0.889     0.941     0.914        17
      sort-2      0.750     1.000     0.857         3
     track-1      0.385     0.833     0.526         6
     track-2      0.385     0.833     0.526         6
     track-3      0.333     0.750     0.462         4
     track-4      0.500     0.750     0.600         4
     track-5      0.500     1.000     0.667         1
     track-6      0.000     0.000     0.000         0
```

```
      artist-4      0.000     0.000     0.000         2
      artist-5      0.000     0.000     0.000         0
      artist-6      0.000     0.000     0.000         0
  music_item-1      0.935     0.935     0.935        31
  music_item-2      1.000     1.000     1.000         3

      accuracy                          0.884       761
     macro avg      0.353     0.370     0.339       761
  weighted avg      0.850     0.884     0.859       761


=== Confusion Matrix ===
                        O       year-1      genre-1      genre-2      genre-3      genre-4
               O      410            0            0            0            0            0
          year-1        1           24            0            0            0            0
         genre-1        0            0            1            0            0            0
         genre-2        0            0            0            1            0            0
         genre-3        0            0            0            0            0            0
         genre-4        0            0            0            0            0            0
         genre-5        0            0            0            0            0            0
         genre-6        0            0            0            0            0            0
       service-1        0            0            0            0            0            0
       service-2        0            0            0            0            0            0
      playlist-1        0            1            0            0            0            0
      playlist-2        0            0            0            0            0            0
      playlist-3        1            0            0            0            0            0
      playlist-4        0            0            0            0            0            0
      playlist-5        1            0            0            0            0            0
      playlist-6        0            0            0            0            0            0
         album-1        1            0            0            0            0            0
         album-2        1            0            0            0            0            0
         album-3        0            0            0            0            0            0
         album-4        0            0            0            0            0            0
         album-5        0            0            0            0            0            0
         album-6        0            0            0            0            0            0
         album-7        0            0            0            0            0            0
         album-8        0            0            0            0            0            0
          sort-1        0            0            0            0            0            0
          sort-2        0            0            0            0            0            0
```

Cell ini menggunakan fungsi train_seq() untuk melatih model CRF menggunakan fitur dari data latih. Model kemudian digunakan untuk memprediksi label entitas pada data validasi yang telah disiapkan sebelumnya. Kinerja model dievaluasi menggunakan F1-Score dan Classification Report, yang menyajikan nilai precision, recall, dan F1-score untuk setiap kategori. Selain itu, Confusion Matrix ditampilkan untuk melihat perbandingan antara label sebenarnya dan hasil prediksi model. Seluruh hasil evaluasi tersebut membantu menilai kemampuan model CRF dalam mengenali entitas pada teks.

05_BERT_Snips
Link Google Colab:
https://colab.research.google.com/drive/1xP7vu8JfLnhkSATUXt2pyxpAQErMMomZ?usp=sharing

Cell 1 – Instalasi dan Import Library untuk Model BERT

```
!pip install transformers pandas scikit-learn matplotlib

import string
import pandas as pd
import numpy as np
import json
from tqdm import tqdm, trange

import torch
from torch.optim import AdamW
from torch.utils.data import TensorDataset, DataLoader, RandomSampler, SequentialSampler
from transformers import BertTokenizer, BertConfig, BertForTokenClassification
from sklearn.model_selection import train_test_split
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.13/dist-packages (5.18.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)
Requirement already satisfied: huggingface-hub<3.0,>=1.31.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (1.33.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.13/dist-packages (from transformers) (2.1.3)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (26.3)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.13/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex>=2025.10.22 in /usr/local/lib/python3.13/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<0.24.0,>=0.23.1 in /usr/local/lib/python3.13/dist-packages (from transformers) (0.23.2)
Requirement already satisfied: typer in /usr/local/lib/python3.13/dist-packages (from transformers) (0.27.2)
Requirement already satisfied: safetensors>=0.8.0 in /usr/local/lib/python3.13/dist-packages (from transformers) (0.8.0)
Requirement already satisfied: tqdm>=4.60 in /usr/local/lib/python3.13/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn) (3.7.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.66.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
```

Cell ini digunakan untuk menginstal dan memanggil berbagai library yang dibutuhkan dalam proses pengolahan teks menggunakan model BERT. Library pandas dan numpy membantu mengelola data, sedangkan torch digunakan untuk membangun dan melatih model deep learning. Selanjutnya, BertTokenizer, BertConfig, dan BertForTokenClassification dari Transformers digunakan untuk memproses teks serta melakukan klasifikasi token, sementara train_test_split membantu membagi dataset menjadi data training dan testing. Library tambahan seperti tqdm digunakan untuk menampilkan perkembangan proses, sedangkan AdamW berfungsi sebagai optimizer untuk memperbarui bobot model selama pelatihan.

Cell 2 – Upload Dataset Training dan Testing

```
try:
    from google.colab import files
    uploaded = files.upload()
    train_loc = "train_PlayMusic_full.json"
    test_loc = "validate_PlayMusic.json"
except ModuleNotFoundError:
    train_loc = "Data/snips/train_PlayMusic_full.json"
    test_loc = "Data/snips/validate_PlayMusic.json"
```

Cell ini digunakan untuk mengunggah file dataset train_PlayMusic_full.json dan validate_PlayMusic.json ke Google Colab. Kedua file tersebut berisi data yang akan digunakan untuk melatih dan menguji model klasifikasi teks. Jika program dijalankan di luar Google Colab, kode akan

menggunakan lokasi file yang sudah disimpan di dalam folder Data/snips. Dengan cara ini, program dapat menyesuaikan lokasi dataset berdasarkan lingkungan tempat kode dijalankan.

Cell 3 – Membaca Dataset dengan format JSON

```
train_file = json.load(open(train_loc, encoding="iso-8859-2"))
test_file = json.load(open(test_loc, encoding="iso-8859-2"))
```

Cell ini digunakan untuk membaca dataset training dan testing dari file JSON yang telah disiapkan sebelumnya. Fungsi json.load() memuat isi file menjadi struktur data Python agar dapat diolah lebih lanjut oleh program. Parameter encoding="iso-8859-2" digunakan untuk membaca file dengan format karakter tersebut. Hasilnya disimpan dalam variabel train_file dan test_file sebagai sumber data untuk tahap pemrosesan berikutnya.

Cell 4 – Mengambil Data teks dari Dataset

```
train_datafile = [i["data"] for i in train_file["PlayMusic"]]
test_datafile = [i["data"] for i in test_file["PlayMusic"]]
```

Cell ini digunakan untuk mengambil bagian data yang terdapat pada kategori PlayMusic dari dataset training dan testing. Program menggunakan perulangan untuk mengakses setiap elemen di dalam kategori tersebut, kemudian mengambil isi bagian data dan menyimpannya ke dalam list. Hasilnya, train_datafile berisi data PlayMusic untuk pelatihan, sedangkan test_datafile berisi data untuk pengujian. Data tersebut selanjutnya dapat diproses untuk menyiapkan teks dan label yang dibutuhkan oleh model BERT.

Cell 5 – Konversi Data Teks menjadi Token dan Label

```
def convert_data(datalist):
    output = []
    for data in datalist:
        sent = []
        pos = []
        for phrase in data:
            words = phrase["text"].strip().split(" ")
            while "" in words:
                words.remove("")
            if not words:
                continue
            if "entity" in phrase.keys():
                label = phrase["entity"]
                labels = [label+"-{}".format(i+1) for i in range(len(words))]
            else:
                labels = ["O"] * len(words)
            sent.extend(words)
            pos.extend(labels)
        if sent:
            output.append([sent, pos])
    return output
```

Cell ini digunakan untuk mengubah data dataset menjadi kumpulan kata (*token*) beserta label yang sesuai untuk kebutuhan pelatihan model BERT. Fungsi convert_data() memisahkan setiap teks menjadi kata-kata menggunakan split(), lalu menghapus bagian kosong agar data lebih rapi. Jika suatu bagian

teks memiliki entity, program akan memberikan label berdasarkan entitas tersebut pada setiap kata, sedangkan kata tanpa entitas diberi label O yang berarti bukan bagian dari entitas. Setelah semua kata dan label dikumpulkan, hasilnya disimpan dalam bentuk pasangan daftar kata dan label untuk digunakan pada tahap pemrosesan selanjutnya.

Cell 6 – Membuat DataFrame untuk Dataset Training dan Testing

```
train_data = convert_data(train_datafile)
test_data = convert_data(test_datafile)

df_train = pd.DataFrame(train_data, columns=['sentence', 'label'])
df_test = pd.DataFrame(test_data, columns=['sentence', 'label'])
df_train.head()
```

	sentence	label
0	[I, need, to, hear, the, song, Aspro, Mavro, f...	[O, O, O, O, O, music_item-1, track-1, track-2...
1	[play, Yo, Ho, from, the, new, york, pops, on,...	[O, track-1, track-2, O, artist-1, artist-2, a...
2	[Play, some, seventies, music, by, Janne, Puur...	[O, O, year-1, O, O, artist-1, artist-2, O, se...
3	[play, the, MĂşsica, Da, SĂŠrie, De, Filmes, O...	[O, O, album-1, album-2, album-3, album-4, alb...
4	[Play, Magic, Sam, from, the, thirties]	[O, artist-1, artist-2, O, O, year-1]

Cell ini digunakan untuk mengubah data yang sudah diproses oleh fungsi convert_data() menjadi DataFrame menggunakan library Pandas. Variabel train_data berisi kalimat dan label untuk pelatihan, sedangkan test_data berisi data untuk pengujian model. Kedua dataset disusun ke dalam dua kolom, yaitu sentence yang berisi kumpulan kata dan label yang berisi label untuk setiap kata. Perintah df_train.head() digunakan untuk menampilkan lima baris pertama data training agar struktur dan hasil pengolahan dataset dapat diperiksa sebelum masuk ke tahap berikutnya.

Cell 7 – Menggabungkan Data dan Menentukan Label Unik

```
sentence = list(df_train['sentence']) + list(df_test['sentence'])
label = list(df_train['label']) + list(df_test['label'])

unique_labels = []
for i in label:
    unique_labels += i

labels = unique_labels
unique_labels = list(set(unique_labels))
unique_labels.append("[PAD]")
list(unique_labels)[:10]
```

```
['genre-4',
 'genre-2',
 'sort-1',
 'album-3',
 'playlist-1',
 'track-8',
 'playlist-5',
 'playlist-2',
 'album-8',
 'genre-1']
```

Cell ini digunakan untuk menggabungkan kalimat dan label dari dataset training serta testing agar seluruh kategori label dapat diketahui. Program kemudian mengumpulkan semua label dari setiap kata ke dalam variabel unique_labels untuk mencari jenis label yang digunakan dalam dataset. Fungsi set()

digunakan untuk menghilangkan label yang duplikat, sehingga setiap label hanya tercatat satu kali. Terakhir, label khusus [PAD] ditambahkan untuk menandai bagian kosong ketika urutan teks disamakan panjangnya, lalu sepuluh label pertama ditampilkan sebagai pemeriksaan awal.

Cell 8 – Menampilkan Contoh Kalimat dari Dataset

```
sentence[:2]
```

```
[['I',
  'need',
  'to',
  'hear',
  'the',
  'song',
  'Aspro',
  'Mavro',
  'from',
  'Bill',
  'Szymczyk',
  'on',
  'Youtube'],
 ['play', 'Yo', 'Ho', 'from', 'the', 'new', 'york', 'pops', 'on', 'Youtube']]
```

Cell ini digunakan untuk menampilkan dua data kalimat pertama yang tersimpan dalam variabel sentence. Perintah sentence[:2] mengambil dua elemen pertama dari daftar tanpa mengubah isi data aslinya. Hasilnya dapat digunakan untuk memeriksa bentuk kalimat yang sudah dikumpulkan dari dataset training dan testing. Dengan melihat contoh ini, kita bisa memastikan bahwa data teks sudah tersedia dan siap digunakan untuk proses tokenisasi menggunakan BERT.

Cell 9 – Mengembalikan Teks ke Bentuk Kalimat yang Rapi

```
import re
def untokenize(words):
    """
    Untokenizing a text undoes the tokenizing operation, restoring
    punctuation and spaces to the places that people expect them to be.
    """
    text = ' '.join(words)
    step1 = text.replace("`` ", '"').replace(" ''", '"').replace('. . .',  '...')
    step2 = step1.replace(" ( ", " (").replace(" ) ", ") ")
    step3 = re.sub(r' ([.,:;?!%]+)([ \'"`])', r"\1\2", step2)
    step4 = re.sub(r' ([.,:;?!%]+)$', r"\1", step3)
    step5 = step4.replace(" '", "'").replace(" n't", "n't").replace(
         "can not", "cannot")
    step6 = step5.replace(" ` ", " '")
    return step6.strip()

sentences_untokenized = [untokenize(i) for i in sentence]
```

Cell ini digunakan untuk merapikan kembali kumpulan kata yang sebelumnya sudah dipisahkan menjadi token agar membentuk kalimat yang lebih alami. Fungsi untokenize() menggabungkan kata-kata menggunakan spasi, kemudian memperbaiki tanda baca, tanda kutip, tanda kurung, dan beberapa bentuk kata seperti can not menjadi cannot. Library re digunakan untuk membantu memperbaiki pola spasi dan tanda baca menggunakan ekspresi reguler. Hasil akhirnya disimpan dalam variabel sentences_untokenized, yang berisi kalimat-kalimat dengan format lebih rapi dan siap digunakan pada tahap pemrosesan berikutnya.

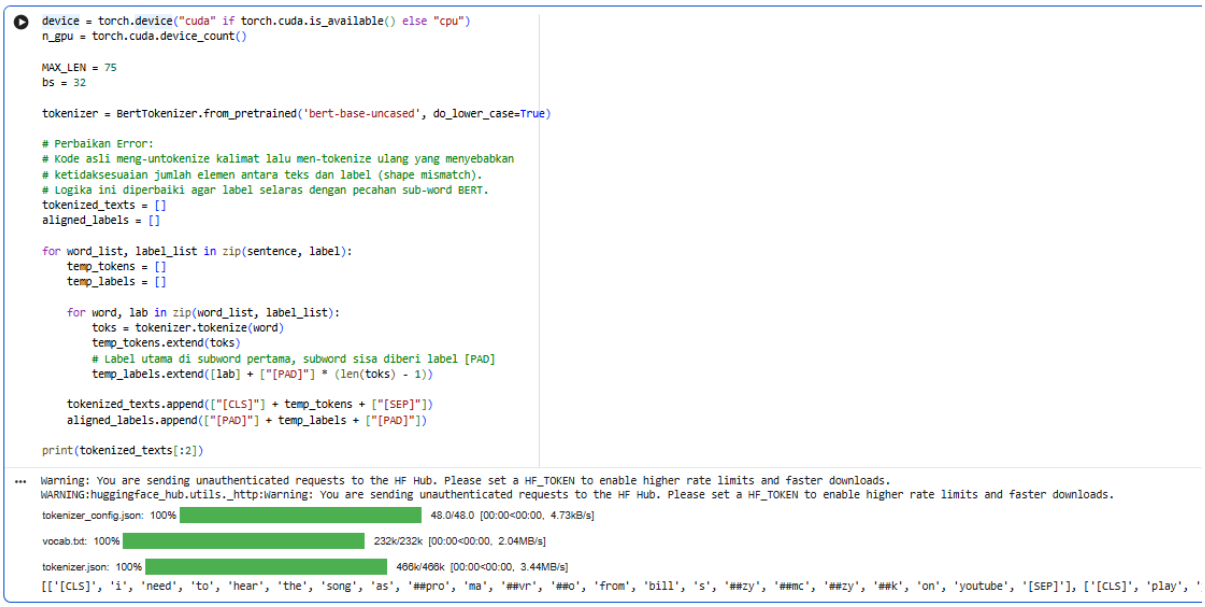
Cell 10 – Menampilkan Contoh Kalimat yang Sudah Dirapikan

```
sentences_untokenized[0]
```

```
'I need to hear the song Aspro Mavro from Bill Szymczyk on Youtube'
```

Cell ini digunakan untuk menampilkan kalimat pertama dari variabel sentences_untokenized yang telah diproses sebelumnya. Perintah sentences_untokenized[0] mengambil elemen pertama dari daftar, dengan indeks dimulai dari angka nol. Output yang ditampilkan membantu kita memeriksa apakah kata-kata dan tanda baca sudah tersusun kembali menjadi kalimat yang lebih rapi dan mudah dibaca. Tahap ini juga memastikan bahwa fungsi untokenize() berjalan sesuai tujuan sebelum data digunakan dalam proses selanjutnya.

Cell 11 – Melakukan Tokenisasi dan Menyelaraskan Label dengan BERT



```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
n_gpu = torch.cuda.device_count()

MAX_LEN = 75
bs = 32

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased', do_lower_case=True)

# Perbaikan Error:
# Kode asli meng-untokenize kalimat lalu men-tokenize ulang yang menyebabkan
# ketidaksesuaian jumlah elemen antara teks dan label (shape mismatch).
# Logika ini diperbaiki agar label selaras dengan pecahan sub-word BERT.
tokenized_texts = []
aligned_labels = []

for word_list, label_list in zip(sentence, label):
    temp_tokens = []
    temp_labels = []

    for word, lab in zip(word_list, label_list):
        toks = tokenizer.tokenize(word)
        temp_tokens.extend(toks)
        # Label utama di subword pertama, subword sisa diberi label [PAD]
        temp_labels.extend([lab] + ["[PAD]"] * (len(toks) - 1))

    tokenized_texts.append(["[CLS]"] + temp_tokens + ["[SEP]"])
    aligned_labels.append(["[PAD]"] + temp_labels + ["[PAD]"])

print(tokenized_texts[:2])
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 4.73kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 2.04MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 3.44MB/s]
[['[CLS]', 'i', 'need', 'to', 'hear', 'the', 'song', 'as', '##pro', 'ma', '##vr', '##o', 'from', 'bill', 's', '##zy', '##mc', '##zy', '##k', 'on', 'youtube', '[SEP]'], ['[CLS]', 'play', '
```

Cell ini digunakan untuk menyiapkan data teks agar dapat diproses oleh model BERT untuk klasifikasi token. Program terlebih dahulu menentukan perangkat yang digunakan, yaitu GPU (CUDA) jika tersedia atau CPU sebagai alternatif, kemudian menetapkan panjang maksimal urutan sebesar 75 token dan batch size sebesar 32. Selanjutnya, BertTokenizer dari model bert-base-uncased memecah setiap kata menjadi token, termasuk subword jika diperlukan, lalu menambahkan token khusus [CLS] di awal dan [SEP] di akhir kalimat. Label juga disesuaikan dengan hasil tokenisasi, yaitu label asli diberikan pada subword pertama, sedangkan subword berikutnya diberi label [PAD] agar jumlah token dan label tetap selaras, kemudian dua contoh hasil tokenisasi ditampilkan untuk diperiksa.

Cell 12 –Memeriksa bentuk dan Ukuran Data Label

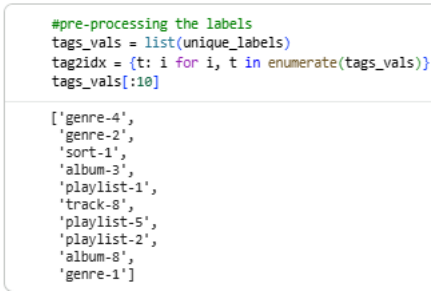
```
np.array(labels).shape
```

```
(15292,)
```

Cell ini digunakan untuk memeriksa bentuk dan ukuran data label yang tersimpan dalam variabel labels menggunakan NumPy. Perintah np.array(labels) mengubah kumpulan label menjadi array NumPy,

sedangkan .shape menampilkan dimensi atau ukuran array tersebut. Pemeriksaan ini membantu mengetahui jumlah label yang tersedia sebelum data diproses lebih lanjut oleh model BERT. Dengan mengetahui bentuk data, kita juga dapat memeriksa kesiapan label untuk disesuaikan dengan token hasil tokenisasi.

Cell 13 – Membuat Pemetaan Label ke Indeks Angka

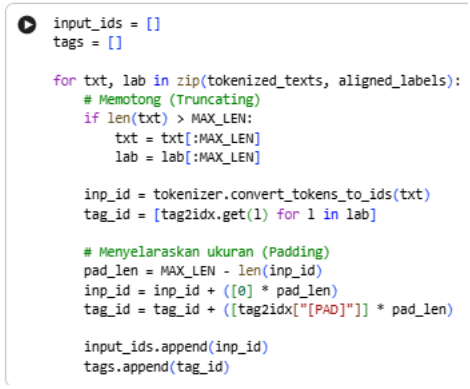


```
#pre-processing the labels
tags_vals = list(unique_labels)
tag2idx = {t: i for i, t in enumerate(tags_vals)}
tags_vals[:10]
```

```
['genre-4',
 'genre-2',
 'sort-1',
 'album-3',
 'playlist-1',
 'track-8',
 'playlist-5',
 'playlist-2',
 'album-8',
 'genre-1']
```

Cell ini digunakan untuk menyiapkan label agar dapat diproses oleh model BERT dalam bentuk angka. Variabel tags_vals menyimpan daftar label unik yang telah dikumpulkan sebelumnya, sedangkan tag2idx membuat pemetaan antara setiap label dan indeks angka menggunakan enumerate(). Pemetaan ini diperlukan karena model membutuhkan representasi numerik untuk mengenali dan memprediksi kategori setiap token. Terakhir, tags_vals[:10] digunakan untuk menampilkan sepuluh label pertama sebagai pemeriksaan hasil persiapan label.

Cell 14 – Mengubah Token dan Label Menjadi Angka serta Padding



```
input_ids = []
tags = []

for txt, lab in zip(tokenized_texts, aligned_labels):
    # Memotong (Truncating)
    if len(txt) > MAX_LEN:
        txt = txt[:MAX_LEN]
        lab = lab[:MAX_LEN]

    inp_id = tokenizer.convert_tokens_to_ids(txt)
    tag_id = [tag2idx.get(l) for l in lab]

    # Menyelaraskan ukuran (Padding)
    pad_len = MAX_LEN - len(inp_id)
    inp_id = inp_id + ([0] * pad_len)
    tag_id = tag_id + ([tag2idx["[PAD]"]] * pad_len)

    input_ids.append(inp_id)
    tags.append(tag_id)
```

Cell ini digunakan untuk mengubah token teks dan label menjadi angka agar bisa diproses oleh model BERT. Fungsi convert_tokens_to_ids() mengubah setiap token menjadi ID numerik, sedangkan tag2idx digunakan untuk mengubah label menjadi indeks angka yang sesuai. Jika panjang kalimat melebihi MAX_LEN sebesar 75 token, data akan dipotong, sedangkan kalimat yang lebih pendek akan ditambahkan padding hingga mencapai panjang yang sama. Hasil akhirnya disimpan dalam variabel input_ids dan tags, sehingga setiap urutan input memiliki panjang yang seragam dan labelnya tetap selaras dengan token yang diproses.

Cell 15 – Membuat Attention Mask untuk Model BERT

```
attention_masks = [[float(i>0) for i in ii] for ii in input_ids]
```

Cell ini digunakan untuk membuat *attention mask* yang membantu model BERT membedakan token asli dengan token tambahan berupa padding. Program memeriksa setiap nilai dalam input_ids menggunakan kondisi i > 0, kemudian mengubah hasilnya menjadi angka 1.0 atau 0.0. Nilai 1.0 menandakan token yang akan diperhatikan oleh model, sedangkan 0.0 menandakan posisi padding yang perlu diabaikan. Hasilnya disimpan dalam variabel attention_masks dan akan digunakan saat proses pelatihan model BERT agar model dapat memproses urutan teks dengan lebih tepat.

Cell 16 – Membagi Data Training dan Validasi

```
tr_inputs, val_inputs, tr_tags, val_tags = train_test_split(
    input_ids, tags, random_state=2018, test_size=0.1
)
tr_masks, val_masks, _, _ = train_test_split(
    attention_masks, input_ids, random_state=2018, test_size=0.1
)
```

Cell ini digunakan untuk membagi dataset menjadi data training dan data validasi sebelum model BERT dilatih. Fungsi train_test_split() membagi input_ids dan tags dengan proporsi 90% untuk training dan 10% untuk validasi, sedangkan random_state=2018 menjaga agar pembagian data tetap konsisten ketika kode dijalankan kembali. Selanjutnya, attention_masks dibagi menggunakan pengaturan yang sama agar mask tetap sesuai dengan data input masing-masing. Hasilnya, tersedia data input, label, dan attention mask untuk proses pelatihan serta evaluasi model.

Cell 17 – Mengubah Data Menjadi Tensor PyTorch

```
tr_inputs = torch.tensor(tr_inputs)
val_inputs = torch.tensor(val_inputs)
tr_tags = torch.tensor(tr_tags)
val_tags = torch.tensor(val_tags)
tr_masks = torch.tensor(tr_masks)
val_masks = torch.tensor(val_masks)
```

Cell ini digunakan untuk mengubah data input, label, dan attention mask dari bentuk list menjadi tensor menggunakan torch.tensor(). Tensor merupakan format data yang dibutuhkan PyTorch agar data dapat diproses oleh model deep learning, termasuk BERT. Data training disimpan dalam variabel tr_inputs, tr_tags, dan tr_masks, sedangkan data validasi disimpan dalam variabel val_inputs, val_tags, dan val_masks. Setelah dikonversi, seluruh data siap digunakan pada tahap berikutnya untuk membangun proses pelatihan dan evaluasi model BERT.

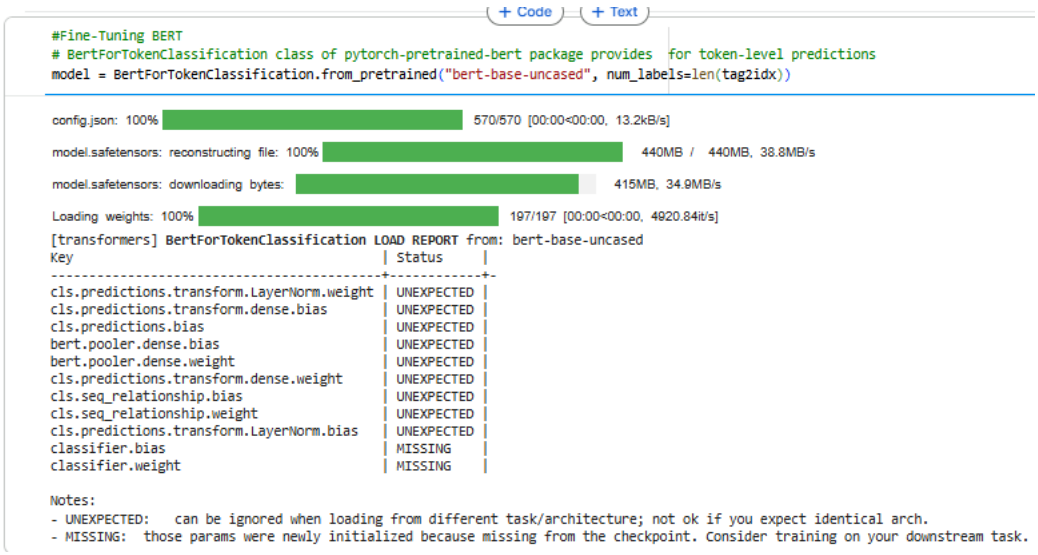
Cell 18 – Menyiapkan DataLoader untuk Training dan Validasi BERT

```
train_data = TensorDataset(tr_inputs, tr_masks, tr_tags)
train_sampler = RandomSampler(train_data)
train_dataloader = DataLoader(train_data, sampler=train_sampler, batch_size=bs)

valid_data = TensorDataset(val_inputs, val_masks, val_tags)
valid_sampler = SequentialSampler(valid_data)
valid_dataloader = DataLoader(valid_data, sampler=valid_sampler, batch_size=bs)
```

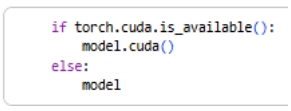
Cell ini digunakan untuk menggabungkan data input, attention mask, dan label ke dalam TensorDataset agar dapat diproses secara berkelompok oleh model BERT. Pada data training, RandomSampler mengacak urutan pengambilan data agar model tidak selalu mempelajari data dengan urutan yang sama. Sementara itu, data validasi menggunakan SequentialSampler agar data diproses secara berurutan saat evaluasi. Terakhir, DataLoader membagi kedua dataset menjadi kelompok-kelompok kecil dengan ukuran batch sebesar 32 sesuai nilai bs, sehingga proses pelatihan dan validasi dapat berjalan lebih teratur.

Cell 19 – Memuat Model BERT untuk Klasifikasi Token (Fine-Tuning)



Cell ini digunakan untuk memuat model BertForTokenClassification dari pretrained model bert-base-uncased yang sudah dilatih sebelumnya menggunakan data teks berbahasa Inggris. Model ini digunakan untuk melakukan klasifikasi pada setiap token dalam kalimat, misalnya menentukan kata yang termasuk ke dalam entitas tertentu atau bukan. Parameter num_labels=len(tag2idx) menentukan jumlah kategori label yang akan diprediksi berdasarkan pemetaan label yang telah dibuat sebelumnya. Model ini kemudian akan melalui proses *fine-tuning*, yaitu menyesuaikan bobot model menggunakan dataset yang digunakan dalam praktikum agar dapat mengenali label sesuai kebutuhan klasifikasi.

Cell 20 – Memindahkan Model BERT ke Perangkat Komputasi



Cell ini digunakan untuk menentukan perangkat yang akan digunakan model BERT selama proses pelatihan. Fungsi torch.cuda.is_available() memeriksa apakah GPU dengan dukungan CUDA tersedia pada lingkungan yang digunakan. Jika tersedia, perintah model.cuda() memindahkan model ke GPU agar proses komputasi dapat berjalan lebih cepat, sedangkan jika tidak tersedia, model tetap berada di CPU. Langkah ini membantu menyesuaikan penggunaan perangkat dengan fasilitas komputasi yang tersedia di Google Colab.

Cell 21- Mengatur Optimizer untuk Fine Tuning Model BERT

```
FULL_FINETUNING = True
if FULL_FINETUNING:
    param_optimizer = list(model.named_parameters())
    no_decay = ['bias', 'gamma', 'beta']
    optimizer_grouped_parameters = [
        {'params': [p for n, p in param_optimizer if not any(nd in n for nd in no_decay)],
         'weight_decay_rate': 0.01},
        {'params': [p for n, p in param_optimizer if any(nd in n for nd in no_decay)],
         'weight_decay_rate': 0.0}
    ]
else:
    param_optimizer = list(model.classifier.named_parameters())
    optimizer_grouped_parameters = [{"params": [p for n, p in param_optimizer]}]

# Perbaikan Error: Menggunakan optimizer AdamW modern
optimizer = AdamW(optimizer_grouped_parameters, lr=3e-5)
```

Cell ini digunakan untuk mengatur proses pembaruan bobot model BERT selama pelatihan atau *fine-tuning*. Variabel FULL_FINETUNING = True menentukan bahwa seluruh parameter model akan dilatih kembali sesuai dengan dataset yang digunakan, bukan hanya layer klasifikasinya. Parameter kemudian dibagi menjadi dua kelompok, yaitu parameter yang menggunakan *weight decay* sebesar 0.01 dan parameter seperti bias yang tidak menggunakan *weight decay*. Terakhir, optimizer AdamW diatur dengan *learning rate* sebesar 3e-5 untuk membantu memperbarui bobot model secara bertahap selama proses pelatihan.

Cell 22 – Membuat Fungsi untuk Menghitung Akurasi Model Bert

```
def flat_accuracy(preds, labels):
    pred_flat = np.argmax(preds, axis=2).flatten()
    labels_flat = labels.flatten()

    mask = labels_flat != tag2idx["[PAD]"]
    return np.sum(pred_flat[mask] == labels_flat[mask]) / len(labels_flat[mask])
```

Cell ini digunakan untuk membuat fungsi flat_accuracy() yang menghitung tingkat ketepatan prediksi model BERT dalam mengklasifikasikan setiap token. Fungsi np.argmax() memilih label dengan nilai prediksi tertinggi untuk setiap token, kemudian hasil prediksi dibandingkan dengan label sebenarnya dari dataset. Sebelum menghitung akurasi, program menggunakan *mask* untuk mengabaikan token berlabel [PAD] agar hasil evaluasi tidak terpengaruh oleh token tambahan yang hanya digunakan untuk menyamakan panjang urutan. Dengan demikian, akurasi yang dihasilkan lebih mencerminkan kemampuan model dalam memprediksi token yang benar-benar digunakan dalam data.

Cell 23 – Melatih dan Mengevaluasi Model BERT

```
epochs = 5
max_grad_norm = 1.0
train_loss_set = []

for _ in trange(epochs, desc="Epoch"):
    # TRAIN loop
    model.train()
    tr_loss = 0
    nb_tr_examples, nb_tr_steps = 0, 0

    for step, batch in enumerate(train_dataloader):
        batch = tuple(t.to(device) for t in batch)
        b_input_ids, b_input_mask, b_labels = batch

        optimizer.zero_grad()

        outputs = model(b_input_ids, token_type_ids=None, attention_mask=b_input_mask, labels=b_labels)
        loss = outputs.loss

        train_loss_set.append(loss.item())
        loss.backward()

        torch.nn.utils.clip_grad_norm_(parameters=model.parameters(), max_norm=max_grad_norm)
        optimizer.step()

        tr_loss += loss.item()
        nb_tr_steps += 1

    print("\nTrain loss: {}".format(tr_loss/nb_tr_steps))

    # VALIDATION loop
    model.eval()
    eval_loss, eval_accuracy = 0, 0
    nb_eval_steps = 0

    for batch in valid_dataloader:
        batch = tuple(t.to(device) for t in batch)
        b_input_ids, b_input_mask, b_labels = batch

        with torch.no_grad():
            outputs = model(b_input_ids, token_type_ids=None, attention_mask=b_input_mask)

        logits = outputs.logits.detach().cpu().numpy()
        label_ids = b_labels.to('cpu').numpy()

        eval_accuracy += flat_accuracy(logits, label_ids)
        nb_eval_steps += 1

    print("Validation Accuracy: {}".format(eval_accuracy/nb_eval_steps))
```

```
Epoch:   0%|          | 0/5 [00:00<?, ?it/s]
Train loss: 0.5591721011325717
Epoch:  20%|██        | 1/5 [00:26<01:45, 26.27s/it]Validation Accuracy: 0.7399602207598409

Train loss: 0.08775091003626585
Epoch:  40%|████      | 2/5 [00:51<01:16, 25.43s/it]Validation Accuracy: 0.8787256544781215

Train loss: 0.048338952008634806
Epoch:  60%|██████    | 3/5 [01:17<00:51, 25.73s/it]Validation Accuracy: 0.9038341737738175

Train loss: 0.033678790740668776
Epoch:  80%|████████  | 4/5 [01:43<00:26, 26.02s/it]Validation Accuracy: 0.9192567215599471

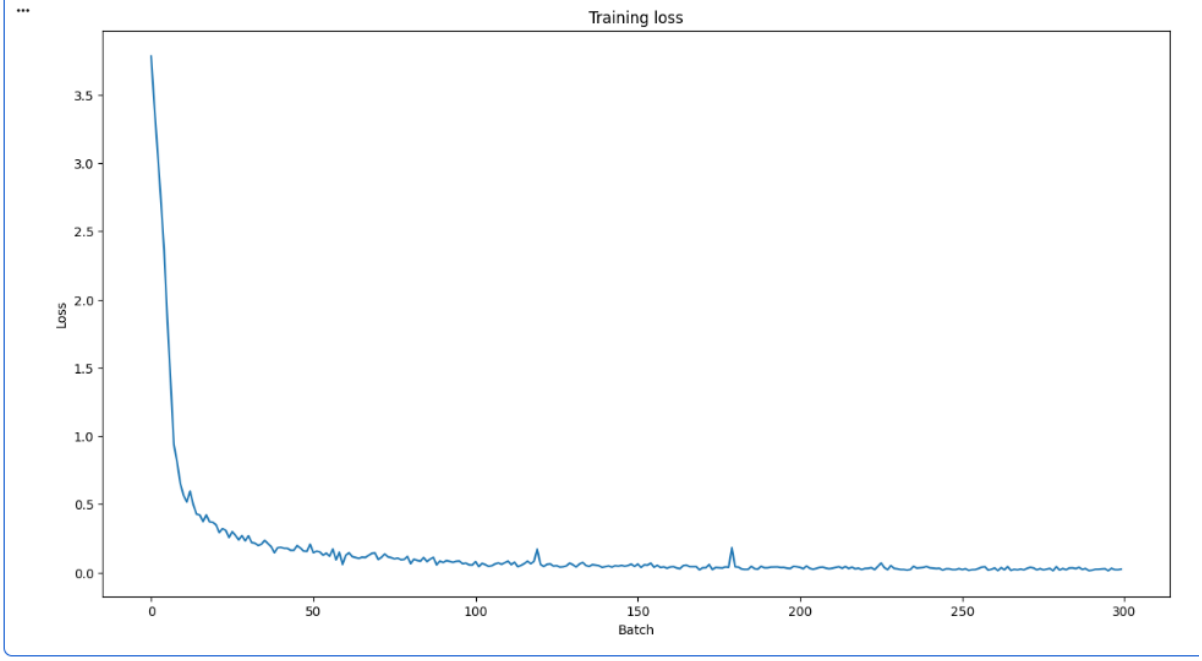
Train loss: 0.02522992758701245
Epoch: 100%|██████████| 5/5 [02:09<00:00, 25.96s/it]Validation Accuracy: 0.9272119453931132
```

Cell ini digunakan untuk melatih model BERT selama 5 epoch agar model dapat mempelajari pola dan mengenali label pada setiap token dalam kalimat. Pada proses training, model menghitung nilai *loss* untuk mengetahui seberapa besar kesalahan prediksi, kemudian memperbarui bobotnya menggunakan optimizer AdamW dan membatasi gradien dengan clip_grad_norm_ agar proses pelatihan lebih stabil. Setelah setiap epoch selesai, model dievaluasi menggunakan data validasi untuk mengukur akurasi prediksinya tanpa memperbarui bobot model. Hasil berupa *training loss* dan *validation accuracy* ditampilkan pada setiap epoch untuk membantu melihat perkembangan performa model selama pelatihan.

Cell 24 – Visualisasi Training Loss Model BERT

```
import matplotlib.pyplot as plt
%matplotlib inline

# plot training performance
plt.figure(figsize=(15,8))
plt.title("Training loss")
plt.xlabel("Batch")
plt.ylabel("Loss")
plt.plot(train_loss_set)
plt.show()
```



Grafik pada cell ini menunjukkan nilai loss selama proses training yang cenderung menurun dari awal hingga akhir. Pada awal training, nilai loss turun dengan sangat cepat, kemudian semakin perlahan dan mendekati nol. Hal ini menunjukkan bahwa model semakin baik dalam mempelajari pola dari data training.